

File Extension

Generated by Doxygen 1.9.8

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

tp		??
--------------------	-------	----

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

tp::ArrowParquetExporter	??
tp::BucketStats	??
tp::BucketStatsBuilder	??
tp::Candle	??
tp::CandleBuilder	??
tp::ColumnarExporter	??
tp::CsvExporter	??
tp::TsvExporter	??
tp::CsvReader	??
tp::CsvWriter	??
tp::FreqEntry	??
tp::FrequencyAnalyzer	??
tp::HeadTail	??
tp::JsonExporter	??
tp::MathResult	??
tp::MathStats	??
tp::MmapTpReader	??
tp::ArrowParquetExporter::Options	??
tp::CsvExporter::Options	??
tp::JsonExporter::Options	??
tp::TpMerger::Options	??
tp::PriceLookup	??
tp::Sequencer	??
tp::StreamTpReader	??
tp::Tick	??
tp::TpDataset	??
tp::TpHeader	??
tp::TpMerger	??
tp::TpReader	??
tp::TpWriter	??

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

tp::ArrowParquetExporter	??
tp::BucketStats	??
tp::BucketStatsBuilder	??
tp::Candle	??
tp::CandleBuilder	??
tp::ColumnarExporter	??
tp::CsvExporter	??
tp::CsvReader	??
tp::CsvWriter	??
tp::FreqEntry	??
tp::FrequencyAnalyzer	??
tp::HeadTail	??
tp::JsonExporter	??
tp::MathResult	??
tp::MathStats	??
tp::MmapTpReader	??
tp::ArrowParquetExporter::Options	??
tp::CsvExporter::Options	??
tp::JsonExporter::Options	??
tp::TpMerger::Options	??
tp::PriceLookup	??
tp::Sequencer	??
tp::StreamTpReader	??
tp::Tick	??
tp::TpDataset	??
tp::TpHeader	??
tp::TpMerger	??
tp::TpReader	??
tp::TpWriter	??
tp::TsvExporter	??

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/arrow_parquet_exporter.h	??
include/bucket_stats.h	??
include/columnar_exporter.h	??
include/csv_exporter.h	??
include/csv_reader.h	??
include/csv_writer.h	??
include/head_tail.h	??
include/json_exporter.h	??
include/merge.h	??
include/mmap_reader.h	??
include/stream_reader.h	??
include/tp_candle.h	??
include/tp_dataset.h	??
include/tp_format.h	??
include/tp_freq.h	??
include/tp_lookup.h	??
include/tp_math.h	??
include/tp_reader.h	??
include/tp_sequence.h	??
include/tp_varint.h	??
include/tp_writer.h	??
include/tsv_exporter.h	??
src/tpcli.cpp	??

Chapter 5

Namespace Documentation

5.1 tp Namespace Reference

Classes

- class [ArrowParquetExporter](#)
- struct [BucketStats](#)
- class [BucketStatsBuilder](#)
- struct [Candle](#)
- class [CandleBuilder](#)
- class [ColumnarExporter](#)
- class [CsvExporter](#)
- class [CsvReader](#)
- class [CsvWriter](#)
- struct [FreqEntry](#)
- class [FrequencyAnalyzer](#)
- class [HeadTail](#)
- class [JsonExporter](#)
- struct [MathResult](#)
- class [MathStats](#)
- class [MmapTpReader](#)
- class [PriceLookup](#)
- class [Sequencer](#)
- class [StreamTpReader](#)
- struct [Tick](#)
- class [TpDataset](#)
- struct [TpHeader](#)
- class [TpMerger](#)
- class [TpReader](#)
- class [TpWriter](#)
- class [TsvExporter](#)

Functions

- void [putVarint](#) (std::ostream &os, uint64_t v)
- uint64_t [getVarint](#) (std::istream &is)
- uint64_t [zigzagEncode](#) (int64_t v)
- int64_t [zigzagDecode](#) (uint64_t v)

Variables

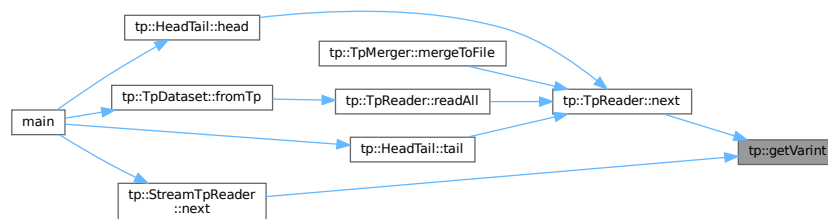
- constexpr char `TP_MAGIC` [4] = {'T','P','O','1'}
- constexpr uint32_t `TP_VERSION` = 1
- constexpr uint32_t `TP_FLAG_COMPRESSED` = 1u << 0

5.1.1 Function Documentation

5.1.1.1 getVarint()

```
uint64_t tp::getVarint (
    std::istream & is ) [inline]
```

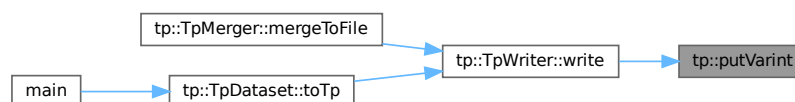
Here is the caller graph for this function:



5.1.1.2 putVarint()

```
void tp::putVarint (
    std::ostream & os,
    uint64_t v ) [inline]
```

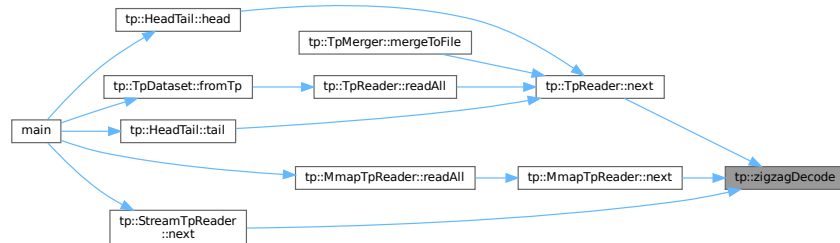
Here is the caller graph for this function:



5.1.1.3 zigzagDecode()

```
int64_t tp::zigzagDecode (
    uint64_t v ) [inline]
```

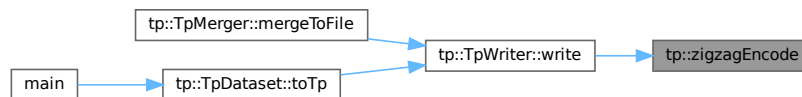
Here is the caller graph for this function:



5.1.1.4 zigzagEncode()

```
uint64_t tp::zigzagEncode (
    int64_t v ) [inline]
```

Here is the caller graph for this function:



5.1.2 Variable Documentation

5.1.2.1 TP_FLAG_COMPRESSED

```
constexpr uint32_t tp::TP_FLAG_COMPRESSED = 1u << 0 [constexpr]
```

5.1.2.2 TP_MAGIC

```
constexpr char tp::TP_MAGIC[4] = {'T','P','0','1'} [constexpr]
```

5.1.2.3 TP_VERSION

```
constexpr uint32_t tp::TP_VERSION = 1 [constexpr]
```

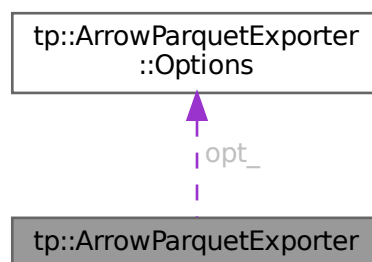

Chapter 6

Class Documentation

6.1 tp::ArrowParquetExporter Class Reference

```
#include <arrow_parquet_exporter.h>
```

Collaboration diagram for tp::ArrowParquetExporter:



Classes

- struct [Options](#)

Public Member Functions

- [ArrowParquetExporter](#) (std::string path)
- [ArrowParquetExporter](#) (std::string path, [Options](#) opt)
- void [write](#) (const std::vector< [Tick](#) > &v) const

Private Attributes

- std::string [path_](#)
- [Options](#) [opt_](#)

6.1.1 Constructor & Destructor Documentation

6.1.1.1 ArrowParquetExporter() [1/2]

```
tp::ArrowParquetExporter::ArrowParquetExporter (
    std::string path ) [inline], [explicit]
```

6.1.1.2 ArrowParquetExporter() [2/2]

```
tp::ArrowParquetExporter::ArrowParquetExporter (
    std::string path,
    Options opt ) [inline], [explicit]
```

6.1.2 Member Function Documentation

6.1.2.1 write()

```
void tp::ArrowParquetExporter::write (
    const std::vector< Tick > & v ) const [inline]
```

Here is the caller graph for this function:



6.1.3 Member Data Documentation

6.1.3.1 opt_

```
Options tp::ArrowParquetExporter::opt_ [private]
```

6.1.3.2 path_

```
std::string tp::ArrowParquetExporter::path_ [private]
```

The documentation for this class was generated from the following file:

- include/[arrow_parquet_exporter.h](#)

6.2 tp::BucketStats Struct Reference

```
#include <bucket_stats.h>
```

Public Attributes

- int64_t [start_time](#) = 0
- int64_t [end_time](#) = 0
- uint64_t [ticks](#) = 0
- double [open](#) = 0
- double [high](#) = 0
- double [low](#) = 0
- double [close](#) = 0
- double [avg_price](#) = 0
- double [sum_price](#) = 0
- double [sum_abs_delta](#) = 0
- double [avg_abs_delta](#) = 0
- double [max_abs_delta](#) = 0
- double [realized_vol](#) = 0
- double [range](#) = 0
- int64_t [min_delta_time](#) = 0
- int64_t [max_delta_time](#) = 0

6.2.1 Member Data Documentation

6.2.1.1 avg_abs_delta

```
double tp::BucketStats::avg_abs_delta = 0
```

6.2.1.2 avg_price

```
double tp::BucketStats::avg_price = 0
```

6.2.1.3 close

```
double tp::BucketStats::close = 0
```

6.2.1.4 end_time

```
int64_t tp::BucketStats::end_time = 0
```

6.2.1.5 high

```
double tp::BucketStats::high = 0
```

6.2.1.6 low

```
double tp::BucketStats::low = 0
```

6.2.1.7 max_abs_delta

```
double tp::BucketStats::max_abs_delta = 0
```

6.2.1.8 max_delta_time

```
int64_t tp::BucketStats::max_delta_time = 0
```

6.2.1.9 min_delta_time

```
int64_t tp::BucketStats::min_delta_time = 0
```

6.2.1.10 open

```
double tp::BucketStats::open = 0
```

6.2.1.11 range

```
double tp::BucketStats::range = 0
```

6.2.1.12 realized_vol

```
double tp::BucketStats::realized_vol = 0
```

6.2.1.13 start_time

```
int64_t tp::BucketStats::start_time = 0
```

6.2.1.14 sum_abs_delta

```
double tp::BucketStats::sum_abs_delta = 0
```

6.2.1.15 sum_price

```
double tp::BucketStats::sum_price = 0
```

6.2.1.16 ticks

```
uint64_t tp::BucketStats::ticks = 0
```

The documentation for this struct was generated from the following file:

- include/[bucket_stats.h](#)

6.3 tp::BucketStatsBuilder Class Reference

```
#include <bucket_stats.h>
```

Static Public Member Functions

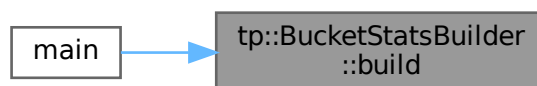
- static std::vector< [BucketStats](#) > [build](#) (const std::vector< [Tick](#) > &v, int64_t intervalSeconds)

6.3.1 Member Function Documentation

6.3.1.1 build()

```
static std::vector< BucketStats > tp::BucketStatsBuilder::build (
    const std::vector< Tick > & v,
    int64_t intervalSeconds ) [inline], [static]
```

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

- include/[bucket_stats.h](#)

6.4 tp::Candle Struct Reference

```
#include <tp_candle.h>
```

Public Attributes

- `int64_t` [start_time](#)
- `int64_t` [end_time](#)
- `double` [open](#)
- `double` [high](#)
- `double` [low](#)
- `double` [close](#)
- `uint64_t` [volume](#)

6.4.1 Member Data Documentation

6.4.1.1 close

```
double tp::Candle::close
```

6.4.1.2 end_time

```
int64_t tp::Candle::end_time
```

6.4.1.3 high

```
double tp::Candle::high
```

6.4.1.4 low

```
double tp::Candle::low
```

6.4.1.5 open

```
double tp::Candle::open
```

6.4.1.6 start_time

```
int64_t tp::Candle::start_time
```

6.4.1.7 volume

```
uint64_t tp::Candle::volume
```

The documentation for this struct was generated from the following file:

- `include/tp_candle.h`

6.5 tp::CandleBuilder Class Reference

```
#include <tp_candle.h>
```

Static Public Member Functions

- static std::vector< [Candle](#) > [build](#) (const std::vector< [Tick](#) > &ticks, int64_t intervalSeconds)

Static Private Member Functions

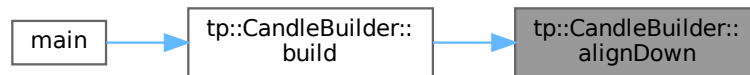
- static int64_t [alignDown](#) (int64_t t, int64_t step)

6.5.1 Member Function Documentation

6.5.1.1 alignDown()

```
static int64_t tp::CandleBuilder::alignDown (
    int64_t t,
    int64_t step ) [inline], [static], [private]
```

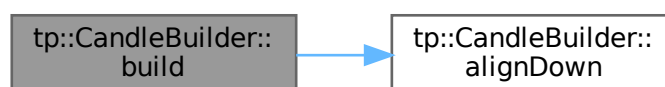
Here is the caller graph for this function:



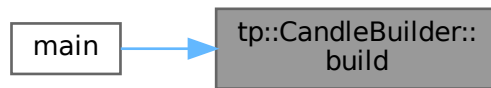
6.5.1.2 build()

```
static std::vector< Candle > tp::CandleBuilder::build (
    const std::vector< Tick > & ticks,
    int64_t intervalSeconds ) [inline], [static]
```

Here is the call graph for this function:



Here is the caller graph for this function:



The documentation for this class was generated from the following file:

- [include/tp_candle.h](#)

6.6 tp::ColumnarExporter Class Reference

```
#include <columnar_exporter.h>
```

Public Member Functions

- [ColumnarExporter](#) (std::string path)
- void [write](#) (const std::vector< [Tick](#) > &v) const

Static Public Attributes

- static constexpr char [COL_MAGIC](#) [4] = {'C','O','L','1'}

Private Attributes

- std::string [path_](#)

6.6.1 Constructor & Destructor Documentation

6.6.1.1 ColumnarExporter()

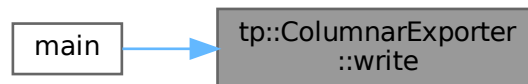
```
tp::ColumnarExporter::ColumnarExporter (  
    std::string path ) [inline], [explicit]
```

6.6.2 Member Function Documentation

6.6.2.1 write()

```
void tp::ColumnarExporter::write (  
    const std::vector< Tick > & v ) const [inline]
```

Here is the caller graph for this function:



6.6.3 Member Data Documentation

6.6.3.1 COL_MAGIC

```
constexpr char tp::ColumnarExporter::COL_MAGIC[4] = {'C','O','L','1'} [static], [constexpr]
```

6.6.3.2 path_

```
std::string tp::ColumnarExporter::path_ [private]
```

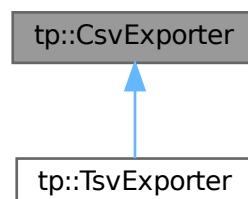
The documentation for this class was generated from the following file:

- include/[columnar_exporter.h](#)

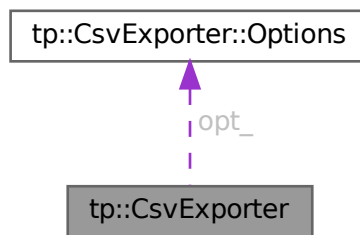
6.7 tp::CsvExporter Class Reference

```
#include <csv_exporter.h>
```

Inheritance diagram for tp::CsvExporter:



Collaboration diagram for `tp::CsvExporter`:



Classes

- struct [Options](#)

Public Member Functions

- [CsvExporter](#) (std::string path)
- [CsvExporter](#) (std::string path, [Options](#) opt)
- void [write](#) (const std::vector< [Tick](#) > &v) const
- template<class It >
void [write](#) (It begin, It end) const

Protected Attributes

- std::string [path_](#)
- [Options](#) [opt_](#)

6.7.1 Constructor & Destructor Documentation

6.7.1.1 CsvExporter() [1/2]

```
tp::CsvExporter::CsvExporter (
    std::string path ) [inline], [explicit]
```

6.7.1.2 CsvExporter() [2/2]

```
tp::CsvExporter::CsvExporter (
    std::string path,
    Options opt ) [inline], [explicit]
```

6.7.2 Member Function Documentation

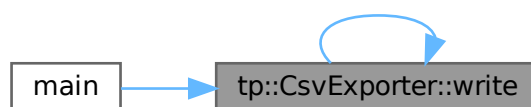
6.7.2.1 write() [1/2]

```
void tp::CsvExporter::write (  
    const std::vector< Tick > & v ) const [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.7.2.2 write() [2/2]

```
template<class It >  
void tp::CsvExporter::write (  
    It begin,  
    It end ) const [inline]
```

6.7.3 Member Data Documentation

6.7.3.1 opt_

```
Options tp::CsvExporter::opt_ [protected]
```

6.7.3.2 path_

```
std::string tp::CsvExporter::path_ [protected]
```

The documentation for this class was generated from the following file:

- include/[csv_exporter.h](#)

6.8 tp::CsvReader Class Reference

```
#include <csv_reader.h>
```

Public Member Functions

- [CsvReader](#) (std::string path)
- std::vector< [Tick](#) > [load](#) ()
- int [maxDecimals](#) () const

Private Member Functions

- void [parseLine](#) (const std::string &line, std::vector< [Tick](#) > &out)

Private Attributes

- std::string [path_](#)
- int [maxDecimals_](#) = 0

6.8.1 Constructor & Destructor Documentation

6.8.1.1 CsvReader()

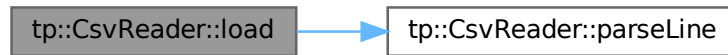
```
tp::CsvReader::CsvReader (  
    std::string path ) [inline], [explicit]
```

6.8.2 Member Function Documentation

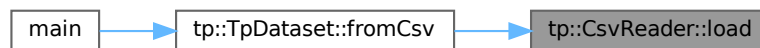
6.8.2.1 load()

```
std::vector< Tick > tp::CsvReader::load ( ) [inline]
```

Here is the call graph for this function:



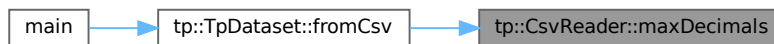
Here is the caller graph for this function:



6.8.2.2 maxDecimals()

```
int tp::CsvReader::maxDecimals ( ) const [inline]
```

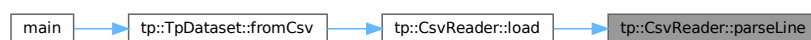
Here is the caller graph for this function:



6.8.2.3 parseLine()

```
void tp::CsvReader::parseLine (
    const std::string & line,
    std::vector< Tick > & out ) [inline], [private]
```

Here is the caller graph for this function:



6.8.3 Member Data Documentation

6.8.3.1 maxDecimals_

```
int tp::CsvReader::maxDecimals_ = 0 [private]
```

6.8.3.2 path_

```
std::string tp::CsvReader::path_ [private]
```

The documentation for this class was generated from the following file:

- include/[csv_reader.h](#)

6.9 tp::CsvWriter Class Reference

```
#include <csv_writer.h>
```

Public Member Functions

- [CsvWriter](#) (std::string path, int decimals=2)
- void [write](#) (const std::vector< [Tick](#) > &ticks, bool withHeader=true) const

Private Attributes

- std::string [path_](#)
- int [decimals_](#)

6.9.1 Constructor & Destructor Documentation

6.9.1.1 CsvWriter()

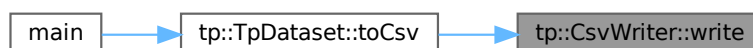
```
tp::CsvWriter::CsvWriter (
    std::string path,
    int decimals = 2 ) [inline], [explicit]
```

6.9.2 Member Function Documentation

6.9.2.1 write()

```
void tp::CsvWriter::write (
    const std::vector< Tick > & ticks,
    bool withHeader = true ) const [inline]
```

Here is the caller graph for this function:



6.9.3 Member Data Documentation

6.9.3.1 decimals_

```
int tp::CsvWriter::decimals_ [private]
```

6.9.3.2 path_

```
std::string tp::CsvWriter::path_ [private]
```

The documentation for this class was generated from the following file:

- include/[csv_writer.h](#)

6.10 tp::FreqEntry Struct Reference

```
#include <tp_freq.h>
```

Public Attributes

- double [price](#)
- uint64_t [count](#)

6.10.1 Member Data Documentation

6.10.1.1 count

```
uint64_t tp::FreqEntry::count
```

6.10.1.2 price

```
double tp::FreqEntry::price
```

The documentation for this struct was generated from the following file:

- include/[tp_freq.h](#)

6.11 tp::FrequencyAnalyzer Class Reference

```
#include <tp_freq.h>
```

Static Public Member Functions

- static std::vector< [FreqEntry](#) > [compute](#) (const std::vector< [Tick](#) > &ticks, uint32_t scale)
- static std::vector< [FreqEntry](#) > [sortedByCountDesc](#) (std::vector< [FreqEntry](#) > v)
- static std::vector< [FreqEntry](#) > [sortedByPrice](#) (std::vector< [FreqEntry](#) > v)

6.11.1 Member Function Documentation

6.11.1.1 compute()

```
static std::vector< FreqEntry > tp::FrequencyAnalyzer::compute (  
    const std::vector< Tick > & ticks,  
    uint32_t scale ) [inline], [static]
```

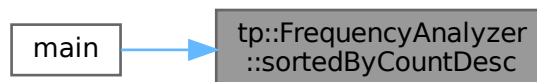
Here is the caller graph for this function:



6.11.1.2 sortedByCountDesc()

```
static std::vector< FreqEntry > tp::FrequencyAnalyzer::sortedByCountDesc (  
    std::vector< FreqEntry > v ) [inline], [static]
```

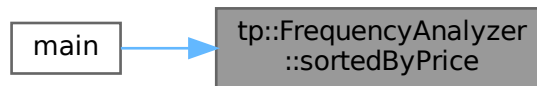
Here is the caller graph for this function:



6.11.1.3 sortedByPrice()

```
static std::vector< FreqEntry > tp::FrequencyAnalyzer::sortedByPrice (
    std::vector< FreqEntry > v ) [inline], [static]
```

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

- [include/tp_freq.h](#)

6.12 tp::HeadTail Class Reference

```
#include <head_tail.h>
```

Static Public Member Functions

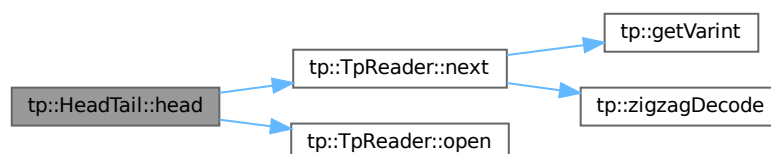
- static std::vector< Tick > [head](#) (const std::string &path, size_t n)
- static std::vector< Tick > [tail](#) (const std::string &path, size_t n)

6.12.1 Member Function Documentation

6.12.1.1 head()

```
static std::vector< Tick > tp::HeadTail::head (
    const std::string & path,
    size_t n ) [inline], [static]
```

Here is the call graph for this function:



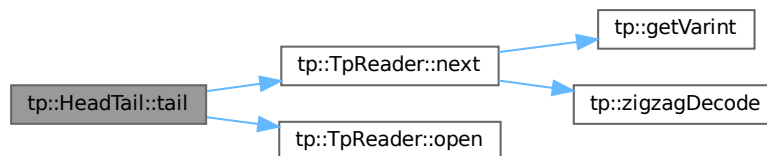
Here is the caller graph for this function:



6.12.1.2 tail()

```
static std::vector< Tick > tp::HeadTail::tail (  
    const std::string & path,  
    size_t n ) [inline], [static]
```

Here is the call graph for this function:



Here is the caller graph for this function:



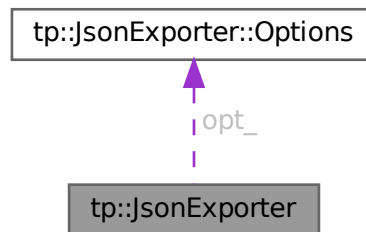
The documentation for this class was generated from the following file:

- [include/head_tail.h](#)

6.13 tp::JsonExporter Class Reference

```
#include <json_exporter.h>
```

Collaboration diagram for tp::JsonExporter:



Classes

- struct [Options](#)

Public Member Functions

- [JsonExporter](#) (std::string path)
- [JsonExporter](#) (std::string path, [Options](#) opt)
- void [write](#) (const std::vector< [Tick](#) > &v) const

Private Attributes

- std::string [path_](#)
- [Options](#) [opt_](#)

6.13.1 Constructor & Destructor Documentation

6.13.1.1 JsonExporter() [1/2]

```
tp::JsonExporter::JsonExporter (
    std::string path ) [inline], [explicit]
```

6.13.1.2 JsonExporter() [2/2]

```
tp::JsonExporter::JsonExporter (
    std::string path,
    Options opt ) [inline], [explicit]
```

6.13.2 Member Function Documentation

6.13.2.1 write()

```
void tp::JsonExporter::write (
    const std::vector< Tick > & v ) const [inline]
```

6.13.3 Member Data Documentation

6.13.3.1 opt_

```
Options tp::JsonExporter::opt_ [private]
```

6.13.3.2 path_

```
std::string tp::JsonExporter::path_ [private]
```

The documentation for this class was generated from the following file:

- [include/json_exporter.h](#)

6.14 tp::MathResult Struct Reference

```
#include <tp_math.h>
```

Public Attributes

- uint64_t [count](#) = 0
- int64_t [min_time](#) = 0
- int64_t [max_time](#) = 0
- double [min_price](#) = 0
- double [max_price](#) = 0
- double [sum](#) = 0
- double [average](#) = 0
- long double [product](#) = 1.0L
- bool [productOverflow](#) = false
- double [geometricMean](#) = 0
- double [sumTimesCount](#) = 0

6.14.1 Member Data Documentation

6.14.1.1 average

```
double tp::MathResult::average = 0
```

6.14.1.2 count

```
uint64_t tp::MathResult::count = 0
```

6.14.1.3 geometricMean

```
double tp::MathResult::geometricMean = 0
```

6.14.1.4 max_price

```
double tp::MathResult::max_price = 0
```

6.14.1.5 max_time

```
int64_t tp::MathResult::max_time = 0
```

6.14.1.6 min_price

```
double tp::MathResult::min_price = 0
```

6.14.1.7 min_time

```
int64_t tp::MathResult::min_time = 0
```

6.14.1.8 product

```
long double tp::MathResult::product = 1.0L
```

6.14.1.9 productOverflow

```
bool tp::MathResult::productOverflow = false
```

6.14.1.10 sum

```
double tp::MathResult::sum = 0
```

6.14.1.11 sumTimesCount

```
double tp::MathResult::sumTimesCount = 0
```

The documentation for this struct was generated from the following file:

- [include/tp_math.h](#)

6.15 tp::MathStats Class Reference

```
#include <tp_math.h>
```

Static Public Member Functions

- static [MathResult compute](#) (const std::vector< [Tick](#) > &ticks)

6.15.1 Member Function Documentation

6.15.1.1 compute()

```
static MathResult tp::MathStats::compute (
    const std::vector< Tick > & ticks ) [inline], [static]
```

Here is the caller graph for this function:



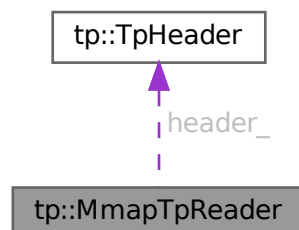
The documentation for this class was generated from the following file:

- [include/tp_math.h](#)

6.16 tp::MmapTpReader Class Reference

```
#include <mmap_reader.h>
```

Collaboration diagram for tp::MmapTpReader:



Public Member Functions

- [MmapTpReader](#) (std::string path)
- [~MmapTpReader](#) ()
- [MmapTpReader](#) (const [MmapTpReader](#) &)=delete
- [MmapTpReader](#) & [operator=](#) (const [MmapTpReader](#) &)=delete
- void [open](#) ()
- void [close](#) ()
- const [TpHeader](#) & [header](#) () const
- bool [next](#) ([Tick](#) &t)
- std::vector< [Tick](#) > [readAll](#) ()

Private Member Functions

- uint64_t [getVarintMem](#) ()

Private Attributes

- std::string [path_](#)
- int [fd_](#) = -1
- const uint8_t * [base_](#) = nullptr
- size_t [size_](#) = 0
- size_t [pos_](#) = 0
- [TpHeader](#) [header_](#) {}
- int64_t [curT_](#) = 0
- int64_t [curP_](#) = 0
- uint64_t [idx_](#) = 0

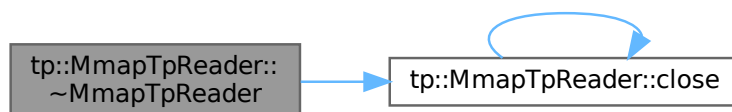
6.16.1 Constructor & Destructor Documentation**6.16.1.1 MmapTpReader() [1/2]**

```
tp::MmapTpReader::MmapTpReader (
    std::string path ) [inline], [explicit]
```

6.16.1.2 ~MmapTpReader()

```
tp::MmapTpReader::~~MmapTpReader ( ) [inline]
```

Here is the call graph for this function:



6.16.1.3 MmapTpReader() [2/2]

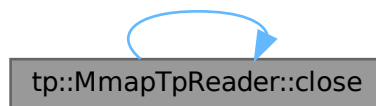
```
tp::MmapTpReader::MmapTpReader (
    const MmapTpReader & ) [delete]
```

6.16.2 Member Function Documentation

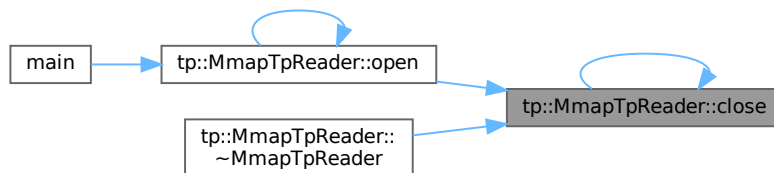
6.16.2.1 close()

```
void tp::MmapTpReader::close ( ) [inline]
```

Here is the call graph for this function:



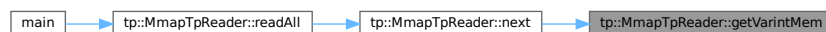
Here is the caller graph for this function:



6.16.2.2 getVarintMem()

```
uint64_t tp::MmapTpReader::getVarintMem ( ) [inline], [private]
```

Here is the caller graph for this function:



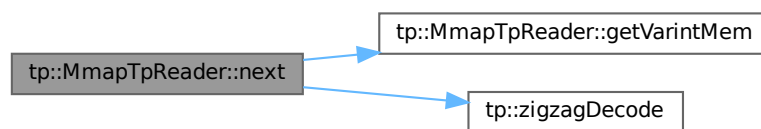
6.16.2.3 header()

```
const TpHeader & tp::MmapTpReader::header ( ) const [inline]
```

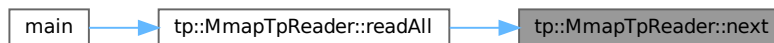
6.16.2.4 next()

```
bool tp::MmapTpReader::next (
    Tick & t ) [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



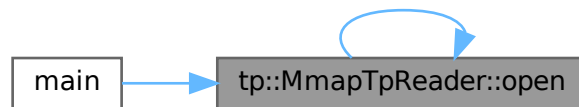
6.16.2.5 open()

```
void tp::MmapTpReader::open ( ) [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.16.2.6 operator=()

```
MmapTpReader & tp::MmapTpReader::operator= (
    const MmapTpReader & ) [delete]
```

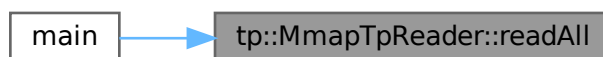
6.16.2.7 readAll()

```
std::vector< Tick > tp::MmapTpReader::readAll ( ) [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.16.3 Member Data Documentation

6.16.3.1 base_

```
const uint8_t* tp::MmapTpReader::base_ = nullptr [private]
```

6.16.3.2 curP_

```
int64_t tp::MmapTpReader::curP_ = 0 [private]
```

6.16.3.3 curT_

```
int64_t tp::MmapTpReader::curT_ = 0 [private]
```

6.16.3.4 fd_

```
int tp::MmapTpReader::fd_ = -1 [private]
```

6.16.3.5 header_

```
TpHeader tp::MmapTpReader::header_ {} [private]
```

6.16.3.6 idx_

```
uint64_t tp::MmapTpReader::idx_ = 0 [private]
```

6.16.3.7 path_

```
std::string tp::MmapTpReader::path_ [private]
```

6.16.3.8 pos_

```
size_t tp::MmapTpReader::pos_ = 0 [private]
```

6.16.3.9 size_

```
size_t tp::MmapTpReader::size_ = 0 [private]
```

The documentation for this class was generated from the following file:

- include/[mmap_reader.h](#)

6.17 tp::ArrowParquetExporter::Options Struct Reference

```
#include <arrow_parquet_exporter.h>
```

Public Attributes

- int [compression_level](#) = 6
- int [row_group_size](#) = 65536

6.17.1 Member Data Documentation

6.17.1.1 [compression_level](#)

```
int tp::ArrowParquetExporter::Options::compression_level = 6
```

6.17.1.2 [row_group_size](#)

```
int tp::ArrowParquetExporter::Options::row_group_size = 65536
```

The documentation for this struct was generated from the following file:

- include/[arrow_parquet_exporter.h](#)

6.18 tp::CsvExporter::Options Struct Reference

```
#include <csv_exporter.h>
```

Public Attributes

- int [decimals](#) = 6
- bool [header](#) = true
- char [sep](#) = ','

6.18.1 Member Data Documentation

6.18.1.1 [decimals](#)

```
int tp::CsvExporter::Options::decimals = 6
```

6.18.1.2 [header](#)

```
bool tp::CsvExporter::Options::header = true
```

6.18.1.3 [sep](#)

```
char tp::CsvExporter::Options::sep = ','
```

The documentation for this struct was generated from the following file:

- include/[csv_exporter.h](#)

6.19 tp::JsonExporter::Options Struct Reference

```
#include <json_exporter.h>
```

Public Attributes

- int [decimals](#) = 6
- bool [pretty](#) = false
- bool [asArrayOfPairs](#) = false
- bool [wrapObject](#) = false

6.19.1 Member Data Documentation

6.19.1.1 asArrayOfPairs

```
bool tp::JsonExporter::Options::asArrayOfPairs = false
```

6.19.1.2 decimals

```
int tp::JsonExporter::Options::decimals = 6
```

6.19.1.3 pretty

```
bool tp::JsonExporter::Options::pretty = false
```

6.19.1.4 wrapObject

```
bool tp::JsonExporter::Options::wrapObject = false
```

The documentation for this struct was generated from the following file:

- [include/json_exporter.h](#)

6.20 tp::TpMerger::Options Struct Reference

```
#include <merge.h>
```

Public Attributes

- bool [dedupe](#) = true
- bool [alreadySorted](#) = false
- int [decimals](#) = 6

6.20.1 Member Data Documentation

6.20.1.1 alreadySorted

```
bool tp::TpMerger::Options::alreadySorted = false
```

6.20.1.2 decimals

```
int tp::TpMerger::Options::decimals = 6
```

6.20.1.3 dedupe

```
bool tp::TpMerger::Options::dedupe = true
```

The documentation for this struct was generated from the following file:

- include/[merge.h](#)

6.21 tp::PriceLookup Class Reference

```
#include <tp_lookup.h>
```

Static Public Member Functions

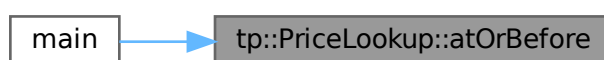
- static const [Tick](#) * [exact](#) (const std::vector< [Tick](#) > &v, int64_t time)
- static const [Tick](#) * [nearest](#) (const std::vector< [Tick](#) > &v, int64_t time)
- static const [Tick](#) * [atOrBefore](#) (const std::vector< [Tick](#) > &v, int64_t time)
- static std::vector< [Tick](#) > [range](#) (const std::vector< [Tick](#) > &v, int64_t t1, int64_t t2)

6.21.1 Member Function Documentation

6.21.1.1 atOrBefore()

```
static const Tick * tp::PriceLookup::atOrBefore (
    const std::vector< Tick > & v,
    int64_t time ) [inline], [static]
```

Here is the caller graph for this function:



6.21.1.2 exact()

```
static const Tick * tp::PriceLookup::exact (
    const std::vector< Tick > & v,
    int64_t time ) [inline], [static]
```

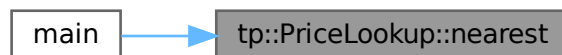
Here is the caller graph for this function:



6.21.1.3 nearest()

```
static const Tick * tp::PriceLookup::nearest (
    const std::vector< Tick > & v,
    int64_t time ) [inline], [static]
```

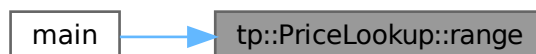
Here is the caller graph for this function:



6.21.1.4 range()

```
static std::vector< Tick > tp::PriceLookup::range (
    const std::vector< Tick > & v,
    int64_t t1,
    int64_t t2 ) [inline], [static]
```

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

- [include/tp_lookup.h](#)

6.22 tp::Sequencer Class Reference

```
#include <tp_sequence.h>
```

Static Public Member Functions

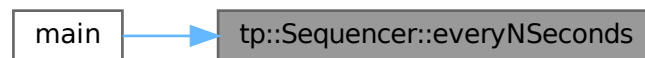
- static std::vector< [Tick](#) > [everyNthTick](#) (const std::vector< [Tick](#) > &v, size_t n)
- static std::vector< [Tick](#) > [everyNSeconds](#) (const std::vector< [Tick](#) > &v, int64_t n)

6.22.1 Member Function Documentation

6.22.1.1 everyNSeconds()

```
static std::vector< Tick > tp::Sequencer::everyNSeconds (
    const std::vector< Tick > & v,
    int64_t n ) [inline], [static]
```

Here is the caller graph for this function:



6.22.1.2 everyNthTick()

```
static std::vector< Tick > tp::Sequencer::everyNthTick (
    const std::vector< Tick > & v,
    size_t n ) [inline], [static]
```

Here is the caller graph for this function:



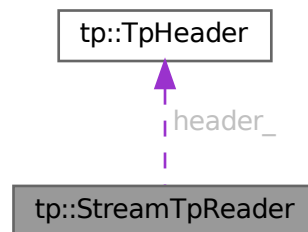
The documentation for this class was generated from the following file:

- [include/tp_sequence.h](#)

6.23 tp::StreamTpReader Class Reference

```
#include <stream_reader.h>
```

Collaboration diagram for tp::StreamTpReader:



Public Member Functions

- [StreamTpReader](#) (std::istream &in)
- void [start](#) ()
- const [TpHeader](#) & [header](#) () const
- bool [next](#) ([Tick](#) &t)

Private Attributes

- std::istream & [in_](#)
- [TpHeader](#) [header_](#) {}
- int64_t [curT_](#) = 0
- int64_t [curP_](#) = 0
- uint64_t [idx_](#) = 0

6.23.1 Constructor & Destructor Documentation

6.23.1.1 StreamTpReader()

```
tp::StreamTpReader::StreamTpReader (
    std::istream & in ) [inline], [explicit]
```

6.23.2 Member Function Documentation

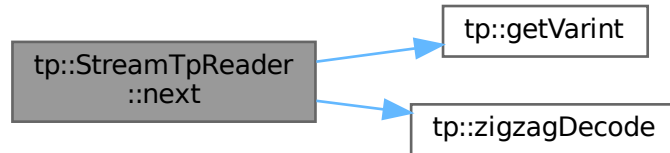
6.23.2.1 header()

```
const TpHeader & tp::StreamTpReader::header ( ) const [inline]
```

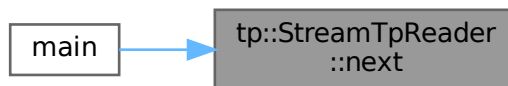
6.23.2.2 next()

```
bool tp::StreamTpReader::next (
    Tick & t ) [inline]
```

Here is the call graph for this function:



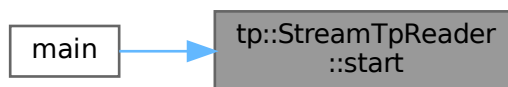
Here is the caller graph for this function:



6.23.2.3 start()

```
void tp::StreamTpReader::start ( ) [inline]
```

Here is the caller graph for this function:



6.23.3 Member Data Documentation

6.23.3.1 curP_

```
int64_t tp::StreamTpReader::curP_ = 0 [private]
```

6.23.3.2 curT_

```
int64_t tp::StreamTpReader::curT_ = 0 [private]
```

6.23.3.3 header_

```
TpHeader tp::StreamTpReader::header_ {} [private]
```

6.23.3.4 idx_

```
uint64_t tp::StreamTpReader::idx_ = 0 [private]
```

6.23.3.5 in_

```
std::istream& tp::StreamTpReader::in_ [private]
```

The documentation for this class was generated from the following file:

- include/[stream_reader.h](#)

6.24 tp::Tick Struct Reference

```
#include <tp_format.h>
```

Public Attributes

- int64_t [time](#)
- double [price](#)

6.24.1 Member Data Documentation

6.24.1.1 price

```
double tp::Tick::price
```

6.24.1.2 time

```
int64_t tp::Tick::time
```

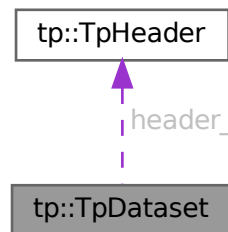
The documentation for this struct was generated from the following file:

- include/[tp_format.h](#)

6.25 tp::TpDataset Class Reference

```
#include <tp_dataset.h>
```

Collaboration diagram for tp::TpDataset:



Public Member Functions

- void [toCsv](#) (const std::string &path) const
- void [toTp](#) (const std::string &path) const
- const [TpHeader](#) & [header](#) () const
- const std::vector< [Tick](#) > & [ticks](#) () const
- size_t [size](#) () const
- bool [empty](#) () const
- int [decimals](#) () const

Static Public Member Functions

- static [TpDataset](#) [fromCsv](#) (const std::string &path)
- static [TpDataset](#) [fromTp](#) (const std::string &path)

Private Member Functions

- void [rebuildHeader](#) ()

Static Private Member Functions

- static int [decimalsFromScale](#) (uint32_t s)

Private Attributes

- [TpHeader](#) [header_](#) {}
- std::vector< [Tick](#) > [ticks_](#)
- int [decimals_](#) = 2

6.25.1 Member Function Documentation

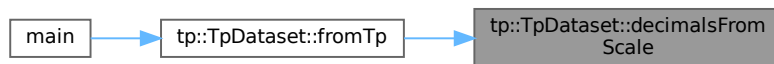
6.25.1.1 decimals()

```
int tp::TpDataset::decimals ( ) const [inline]
```

6.25.1.2 decimalsFromScale()

```
static int tp::TpDataset::decimalsFromScale (
    uint32_t s ) [inline], [static], [private]
```

Here is the caller graph for this function:



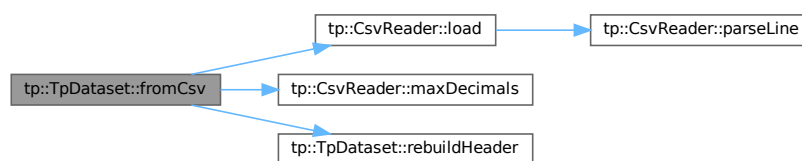
6.25.1.3 empty()

```
bool tp::TpDataset::empty ( ) const [inline]
```

6.25.1.4 fromCsv()

```
static TpDataset tp::TpDataset::fromCsv (
    const std::string & path ) [inline], [static]
```

Here is the call graph for this function:



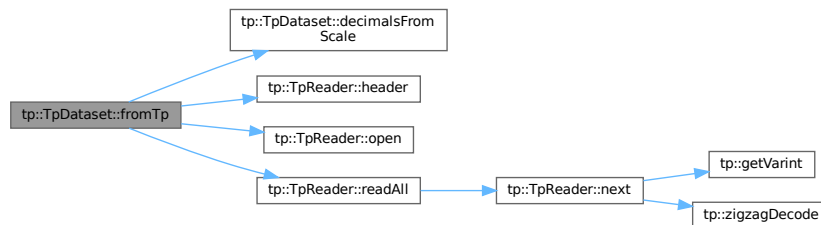
Here is the caller graph for this function:



6.25.1.5 fromTp()

```
static TpDataset tp::TpDataset::fromTp (
    const std::string & path ) [inline], [static]
```

Here is the call graph for this function:



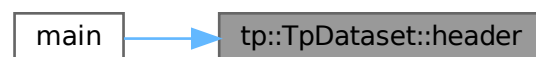
Here is the caller graph for this function:



6.25.1.6 header()

```
const TpHeader & tp::TpDataset::header ( ) const [inline]
```

Here is the caller graph for this function:



6.25.1.7 rebuildHeader()

```
void tp::TpDataset::rebuildHeader ( ) [inline], [private]
```

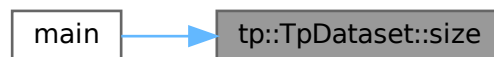
Here is the caller graph for this function:



6.25.1.8 size()

```
size_t tp::TpDataset::size ( ) const [inline]
```

Here is the caller graph for this function:



6.25.1.9 ticks()

```
const std::vector< Tick > & tp::TpDataset::ticks ( ) const [inline]
```

Here is the caller graph for this function:



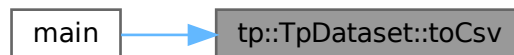
6.25.1.10 toCsv()

```
void tp::TpDataset::toCsv (
    const std::string & path ) const [inline]
```

Here is the call graph for this function:



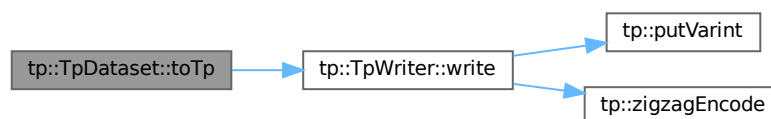
Here is the caller graph for this function:



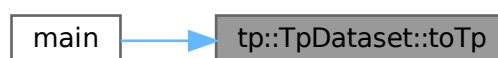
6.25.1.11 toTp()

```
void tp::TpDataset::toTp (
    const std::string & path ) const [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.25.2 Member Data Documentation

6.25.2.1 decimals_

```
int tp::TpDataset::decimals_ = 2 [private]
```

6.25.2.2 header_

```
TpHeader tp::TpDataset::header_ {} [private]
```

6.25.2.3 ticks_

```
std::vector<Tick> tp::TpDataset::ticks_ [private]
```

The documentation for this class was generated from the following file:

- include/[tp_dataset.h](#)

6.26 tp::TpHeader Struct Reference

```
#include <tp_format.h>
```

Public Attributes

- char [magic](#) [4]
- uint32_t [version](#)
- uint32_t [flags](#)
- int64_t [start_time](#)
- int64_t [end_time](#)
- double [min_price](#)
- double [max_price](#)
- uint64_t [count](#)
- uint32_t [price_scale](#)
- uint32_t [reserved](#)

6.26.1 Member Data Documentation

6.26.1.1 count

```
uint64_t tp::TpHeader::count
```

6.26.1.2 end_time

```
int64_t tp::TpHeader::end_time
```

6.26.1.3 flags

```
uint32_t tp::TpHeader::flags
```

6.26.1.4 magic

```
char tp::TpHeader::magic[4]
```

6.26.1.5 max_price

```
double tp::TpHeader::max_price
```

6.26.1.6 min_price

```
double tp::TpHeader::min_price
```

6.26.1.7 price_scale

```
uint32_t tp::TpHeader::price_scale
```

6.26.1.8 reserved

```
uint32_t tp::TpHeader::reserved
```

6.26.1.9 start_time

```
int64_t tp::TpHeader::start_time
```

6.26.1.10 version

```
uint32_t tp::TpHeader::version
```

The documentation for this struct was generated from the following file:

- [include/tp_format.h](#)

6.27 tp::TpMerger Class Reference

```
#include <merge.h>
```

Classes

- struct [Options](#)

Static Public Member Functions

- static void [mergeToFile](#) (const std::vector< std::string > &inputs, const std::string &output)
- static void [mergeToFile](#) (const std::vector< std::string > &inputs, const std::string &output, [Options](#) opt)

6.27.1 Member Function Documentation

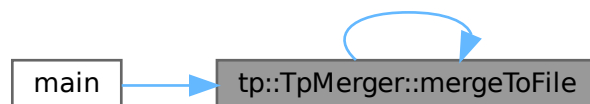
6.27.1.1 mergeToFile() [1/2]

```
static void tp::TpMerger::mergeToFile (  
    const std::vector< std::string > & inputs,  
    const std::string & output ) [inline], [static]
```

Here is the call graph for this function:



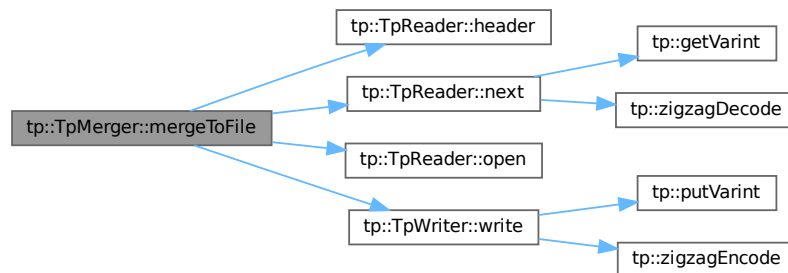
Here is the caller graph for this function:



6.27.1.2 mergeToFile() [2/2]

```
static void tp::TpMerger::mergeToFile (
    const std::vector< std::string > & inputs,
    const std::string & output,
    Options opt ) [inline], [static]
```

Here is the call graph for this function:



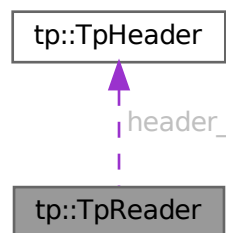
The documentation for this class was generated from the following file:

- include/[merge.h](#)

6.28 tp::TpReader Class Reference

```
#include <tp_reader.h>
```

Collaboration diagram for `tp::TpReader`:



Public Member Functions

- [TpReader](#) (std::string path)
- void [open](#) ()
- const [TpHeader](#) & [header](#) () const
- bool [next](#) (Tick &outTick)
- std::vector< [Tick](#) > [readAll](#) ()

Private Attributes

- std::string [path_](#)
- std::ifstream [in_](#)
- [TpHeader](#) [header_](#) {}
- int64_t [curT_](#) = 0
- int64_t [curP_](#) = 0
- uint64_t [idx_](#) = 0

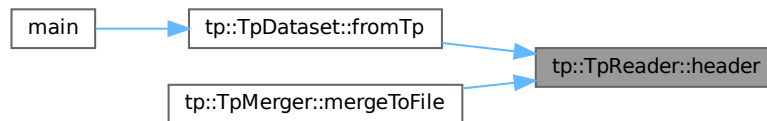
6.28.1 Constructor & Destructor Documentation**6.28.1.1 TpReader()**

```
tp::TpReader::TpReader (
    std::string path ) [inline], [explicit]
```

6.28.2 Member Function Documentation**6.28.2.1 header()**

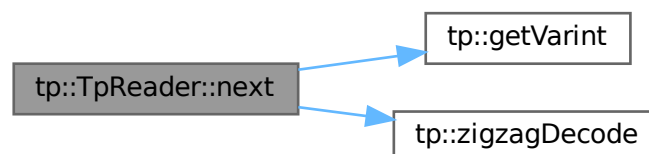
```
const TpHeader & tp::TpReader::header ( ) const [inline]
```

Here is the caller graph for this function:

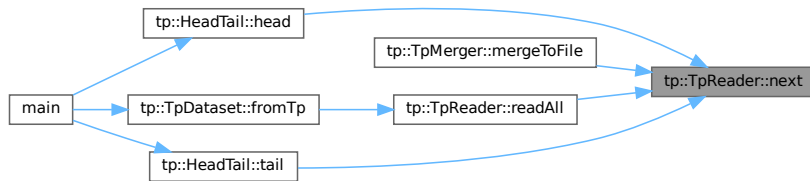
**6.28.2.2 next()**

```
bool tp::TpReader::next (
    Tick & outTick ) [inline]
```

Here is the call graph for this function:



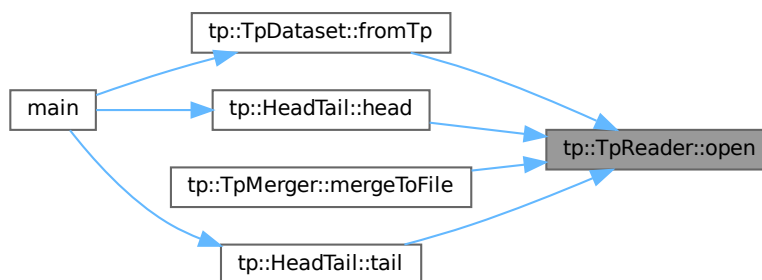
Here is the caller graph for this function:



6.28.2.3 open()

```
void tp::TpReader::open ( ) [inline]
```

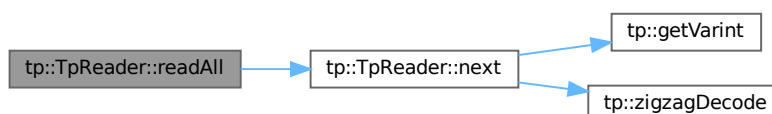
Here is the caller graph for this function:



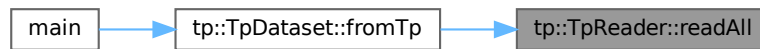
6.28.2.4 readAll()

```
std::vector< Tick > tp::TpReader::readAll ( ) [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.28.3 Member Data Documentation

6.28.3.1 curP_

```
int64_t tp::TpReader::curP_ = 0 [private]
```

6.28.3.2 curT_

```
int64_t tp::TpReader::curT_ = 0 [private]
```

6.28.3.3 header_

```
TpHeader tp::TpReader::header_ {} [private]
```

6.28.3.4 idx_

```
uint64_t tp::TpReader::idx_ = 0 [private]
```

6.28.3.5 in_

```
std::ifstream tp::TpReader::in_ [private]
```

6.28.3.6 path_

```
std::string tp::TpReader::path_ [private]
```

The documentation for this class was generated from the following file:

- [include/tp_reader.h](#)

6.29 tp::TpWriter Class Reference

```
#include <tp_writer.h>
```

Public Member Functions

- [TpWriter](#) (std::string path)
- void [write](#) (const std::vector< [Tick](#) > &ticks, int decimals=2) const

Private Attributes

- std::string [path_](#)

6.29.1 Constructor & Destructor Documentation

6.29.1.1 TpWriter()

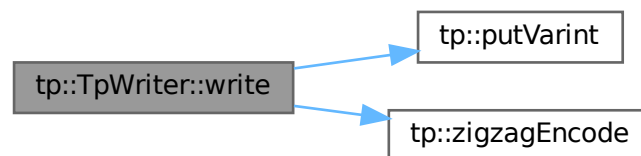
```
tp::TpWriter::TpWriter (
    std::string path ) [inline], [explicit]
```

6.29.2 Member Function Documentation

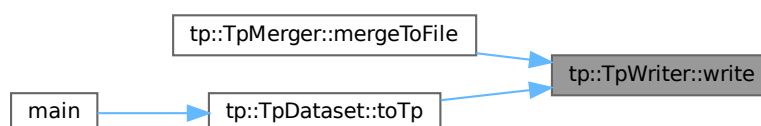
6.29.2.1 write()

```
void tp::TpWriter::write (
    const std::vector< Tick > & ticks,
    int decimals = 2 ) const [inline]
```

Here is the call graph for this function:



Here is the caller graph for this function:



6.29.3 Member Data Documentation

6.29.3.1 path_

```
std::string tp::TpWriter::path_ [private]
```

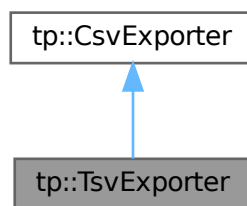
The documentation for this class was generated from the following file:

- [include/tp_writer.h](#)

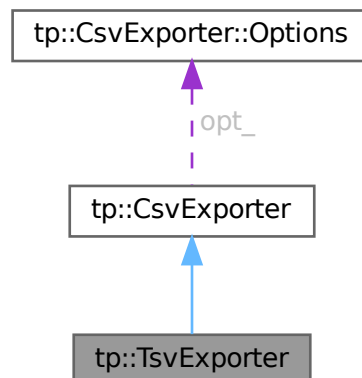
6.30 tp::TsvExporter Class Reference

```
#include <tsv_exporter.h>
```

Inheritance diagram for tp::TsvExporter:



Collaboration diagram for tp::TsvExporter:



Public Member Functions

- [TsvExporter](#) (std::string path, [Options](#) opt={})

Public Member Functions inherited from [tp::CsvExporter](#)

- [CsvExporter](#) (std::string path)
- [CsvExporter](#) (std::string path, [Options](#) opt)
- void [write](#) (const std::vector< [Tick](#) > &v) const
- template<class It >
void [write](#) (It begin, It end) const

Static Private Member Functions

- static [Options](#) [fix](#) ([Options](#) o)

Additional Inherited Members

Protected Attributes inherited from [tp::CsvExporter](#)

- std::string [path_](#)
- [Options](#) [opt_](#)

6.30.1 Constructor & Destructor Documentation

6.30.1.1 TsvExporter()

```
tp::TsvExporter::TsvExporter (
    std::string path,
    Options opt = {} ) [inline], [explicit]
```

6.30.2 Member Function Documentation

6.30.2.1 fix()

```
static Options tp::TsvExporter::fix (
    Options o ) [inline], [static], [private]
```

The documentation for this class was generated from the following file:

- include/[tsv_exporter.h](#)

Chapter 7

File Documentation

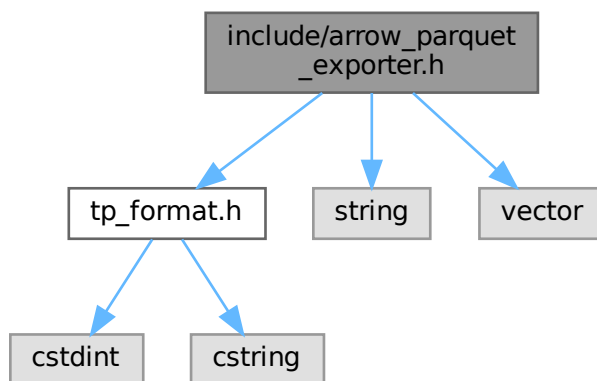
7.1 include/arrow_parquet_exporter.h File Reference

```
#include "tp_format.h"
```

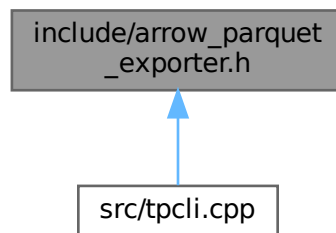
```
#include <string>
```

```
#include <vector>
```

Include dependency graph for arrow_parquet_exporter.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [tp::ArrowParquetExporter](#)
- struct [tp::ArrowParquetExporter::Options](#)

Namespaces

- namespace [tp](#)

7.2 arrow_parquet_exporter.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 // -----
00003 // Real Parquet via Apache Arrow. Compile with:
00004 //     -DTP_HAVE_ARROW
00005 //     -I/path/to/arrow/include
00006 //     -larrow -lparquet
00007 // If TP_HAVE_ARROW is not defined, this class is a no-op stub so the rest of
00008 // the project still compiles.
00009 // -----
00010 #include "tp_format.h"
00011 #include <string>
00012 #include <vector>
00013
00014 #ifdef TP_HAVE_ARROW
00015     #include <arrow/api.h>
00016     #include <arrow/io/file.h>
00017     #include <parquet/arrow/writer.h>
00018 #endif
00019
00020 namespace tp {
00021
00022 class ArrowParquetExporter {
00023 public:
00024     struct Options {
00025         int compression_level = 6;
00026         int row_group_size    = 65536;
00027     };
00028
00029     explicit ArrowParquetExporter(std::string path)
00030         : path_(std::move(path)), opt_{} {}
00031
00032     explicit ArrowParquetExporter(std::string path, Options opt)
00033         : path_(std::move(path)), opt_(opt) {}
00034
00035     void write(const std::vector<Tick>& v) const;
  
```

```

00036
00037 private:
00038     std::string path_;
00039     Options opt_;
00040 };
00041
00042 #ifdef TP_HAVE_ARROW
00043 // ---- definition only when Arrow is available ----
00044 inline void ArrowParquetExporter::write(const std::vector<Tick>& v) const {
00045     arrow::Int64Builder tbuilder;
00046     arrow::DoubleBuilder pbuilder;
00047     (void)tbuilder.Reserve(v.size());
00048     (void)pbuilder.Reserve(v.size());
00049     for (auto& t : v) {
00050         (void)tbuilder.Append(t.time);
00051         (void)pbuilder.Append(t.price);
00052     }
00053     std::shared_ptr<arrow::Array> ta, pa;
00054     (void)tbuilder.Finish(&ta);
00055     (void)pbuilder.Finish(&pa);
00056
00057     auto schema = arrow::schema({
00058         arrow::field("time", arrow::int64()),
00059         arrow::field("price", arrow::float64()),
00060     });
00061     auto table = arrow::Table::Make(schema, {ta, pa});
00062
00063     auto outfile = arrow::io::FileOutputStream::Open(path_.ValueOrDie());
00064     parquet::WriterProperties::Builder b;
00065     b.compression(parquet::Compression::SNAPPY);
00066     b.max_row_group_length(opt_.row_group_size);
00067     parquet::arrow::WriteTable(*table, arrow::default_memory_pool(),
00068                               outfile, opt_.row_group_size, b.build());
00069 }
00070 #else
00071 inline void ArrowParquetExporter::write(const std::vector<Tick>&) const {
00072     throw std::runtime_error(
00073         "ArrowParquetExporter: rebuild with -DTP_HAVE_ARROW and link "
00074         "-larrow -lparquet to enable Parquet output.");
00075 }
00076 #endif
00077
00078 } // namespace tp

```

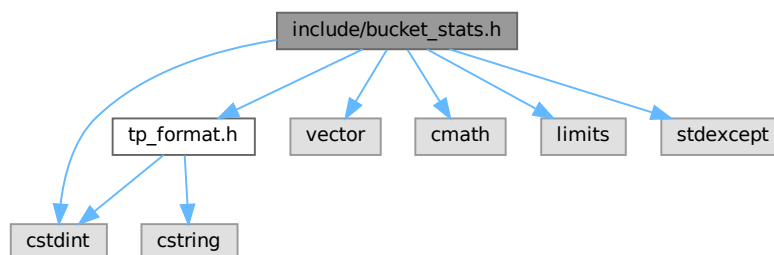
7.3 include/bucket_stats.h File Reference

```

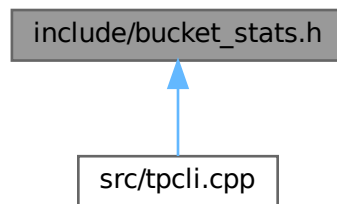
#include "tp_format.h"
#include <vector>
#include <cmath>
#include <cstdint>
#include <limits>
#include <stdexcept>

```

Include dependency graph for bucket_stats.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [tp::BucketStats](#)
- class [tp::BucketStatsBuilder](#)

Namespaces

- namespace [tp](#)

7.4 bucket_stats.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include <vector>
00004 #include <cmath>
00005 #include <stdint>
00006 #include <limits>
00007 #include <stdexcept>
00008
00009 namespace tp {
00010
00011 struct BucketStats {
00012     int64_t start_time = 0;
00013     int64_t end_time   = 0;
00014     uint64_t ticks     = 0;
00015     double open        = 0;
00016     double high        = 0;
00017     double low         = 0;
00018     double close       = 0;
00019     double avg_price   = 0;
00020     double sum_price   = 0;
00021     double sum_abs_delta = 0; // |price| between consecutive ticks
00022     double avg_abs_delta = 0;
00023     double max_abs_delta = 0;
00024     double realized_vol  = 0; // sqrt( (price)^2)
00025     double range         = 0; // high - low
00026     int64_t min_delta_time = 0;
00027     int64_t max_delta_time = 0;
00028 };
00029
00030 class BucketStatsBuilder {
00031 public:
00032     // intervalSeconds > 0. Buckets aligned to epoch.
00033     static std::vector<BucketStats> build(const std::vector<Tick>& v,
00034                                           int64_t intervalSeconds)
00035     {
00036         if (intervalSeconds <= 0) throw std::runtime_error("interval must be > 0");
00037         std::vector<BucketStats> out;
  
```

```

00038         if (v.empty()) return out;
00039
00040         auto align = [](int64_t t, int64_t s) {
00041             int64_t r = t % s; if (r < 0) r += s; return t - r;
00042         };
00043
00044         int64_t bucket = align(v.front().time, intervalSeconds);
00045         BucketStats b;
00046         auto reset = [&](const Tick& t, int64_t bk) {
00047             b = BucketStats{};
00048             b.start_time = bk;
00049             b.end_time   = bk + intervalSeconds;
00050             b.open = b.high = b.low = b.close = t.price;
00051             b.min_delta_time = std::numeric_limits<int64_t>::max();
00052             b.max_delta_time = std::numeric_limits<int64_t>::min();
00053         };
00054         reset(v.front(), bucket);
00055
00056         bool havePrev = false;
00057         int64_t prevTime = 0;
00058         double prevPrice = 0;
00059
00060         for (const auto& t : v) {
00061             int64_t bk = align(t.time, intervalSeconds);
00062             if (bk != bucket) {
00063                 if (b.ticks) out.push_back(b);
00064                 bucket = bk;
00065                 reset(t, bk);
00066                 // reset delta-chain across bucket boundary
00067                 havePrev = false;
00068             }
00069
00070             if (t.price > b.high) b.high = t.price;
00071             if (t.price < b.low) b.low = t.price;
00072             b.close = t.price;
00073             b.sum_price += t.price;
00074             ++b.ticks;
00075
00076             if (havePrev) {
00077                 double d = std::fabs(t.price - prevPrice);
00078                 int64_t dt = t.time - prevTime;
00079                 b.sum_abs_delta += d;
00080                 b.realized_vol += d * d;
00081                 if (d > b.max_abs_delta) b.max_abs_delta = d;
00082                 if (dt < b.min_delta_time) b.min_delta_time = dt;
00083                 if (dt > b.max_delta_time) b.max_delta_time = dt;
00084             }
00085             prevTime = t.time;
00086             prevPrice = t.price;
00087             havePrev = true;
00088         }
00089         if (b.ticks) out.push_back(b);
00090
00091         for (auto& x : out) {
00092             x.avg_price = x.sum_price / double(x.ticks);
00093             x.avg_abs_delta = x.ticks > 1 ? x.sum_abs_delta / double(x.ticks - 1) : 0.0;
00094             x.realized_vol = std::sqrt(x.realized_vol);
00095             x.range = x.high - x.low;
00096         }
00097         return out;
00098     }
00099 };
00100
00101 } // namespace tp

```

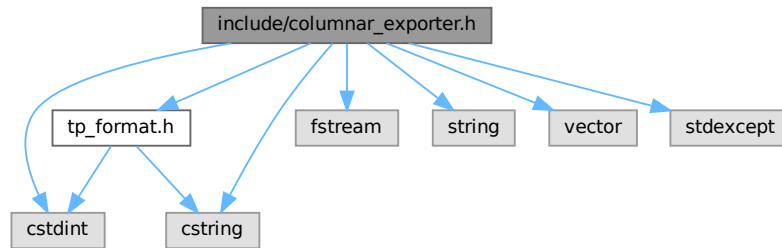
7.5 include/columnar_exporter.h File Reference

```

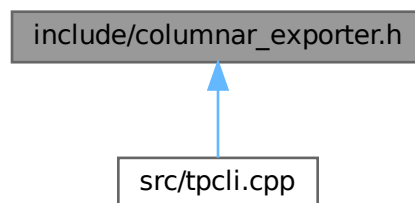
#include "tp_format.h"
#include <fstream>
#include <cstring>
#include <string>
#include <vector>
#include <cstdint>
#include <stdexcept>

```

Include dependency graph for `columnar_exporter.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class `tp::ColumnarExporter`

Namespaces

- namespace `tp`

7.6 columnar_exporter.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include <fstream>
00004 #include <cstring>
00005 #include <string>
00006 #include <vector>
00007 #include <cstdint>
00008 #include <stdexcept>
00009
00010 namespace tp {
00011
00012 // -----
  
```

```

00013 // Simple self-describing columnar binary (.col).
00014 // Fast to write, fast to memmap, easy to parse. Not Parquet spec-compliant.
00015 //
00016 // Layout:
00017 //   magic      "COL1"                (4)
00018 //   version    uint32                 (4)
00019 //   count      uint64                 (8)
00020 //   time_col   int64[count]           (8*N, LE)
00021 //   price_col  double[count]          (8*N, LE, IEEE754)
00022 // -----
00023 class ColumnarExporter {
00024 public:
00025     static constexpr char COL_MAGIC[4] = {'C', 'O', 'L', '1'};
00026
00027     explicit ColumnarExporter(std::string path) : path_(std::move(path)) {}
00028
00029     void write(const std::vector<Tick>& v) const {
00030         std::ofstream out(path_, std::ios::binary);
00031         if (!out) throw std::runtime_error("Cannot create " + path_);
00032
00033         const uint32_t ver = 1;
00034         const uint64_t count = v.size();
00035
00036         out.write(COL_MAGIC, 4);
00037         out.write(reinterpret_cast<const char*>(&ver), 4);
00038         out.write(reinterpret_cast<const char*>(&count), 8);
00039
00040         for (auto& t : v) out.write(reinterpret_cast<const char*>(&t.time), 8);
00041         for (auto& t : v) out.write(reinterpret_cast<const char*>(&t.price), 8);
00042     }
00043
00044 private:
00045     std::string path_;
00046 };
00047
00048 } // namespace tp

```

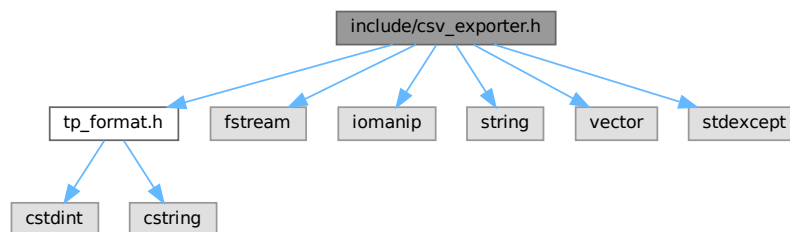
7.7 include/csv_exporter.h File Reference

```

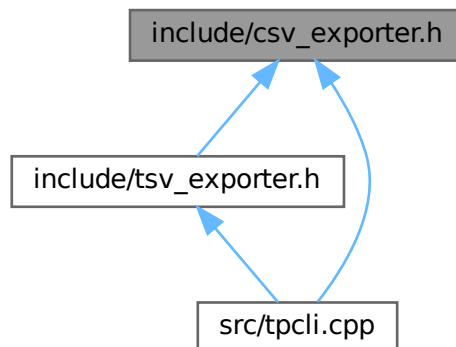
#include "tp_format.h"
#include <fstream>
#include <iomanip>
#include <string>
#include <vector>
#include <stdexcept>

```

Include dependency graph for csv_exporter.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [tp::CsvExporter](#)
- struct [tp::CsvExporter::Options](#)

Namespaces

- namespace [tp](#)

7.8 csv_exporter.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include <fstream>
00004 #include <iomanip>
00005 #include <string>
00006 #include <vector>
00007 #include <stdexcept>
00008
00009 namespace tp {
00010
00011 class CsvExporter {
00012 public:
00013     struct Options {
00014         int decimals = 6;
00015         bool header = true;
00016         char sep = ',';
00017     };
00018
00019     explicit CsvExporter(std::string path)
00020         : path_(std::move(path)), opt_{} {}
00021
00022     explicit CsvExporter(std::string path, Options opt)
00023         : path_(std::move(path)), opt_(opt) {}
00024
00025     void write(const std::vector<Tick>& v) const { write(v.begin(), v.end()); }
00026
00027     // Streaming overload -- pipe a HeadTail/Sequencer/Reader directly
00028     template <class It>
00029     void write(It begin, It end) const {
00030         std::ofstream out(path_);
  
```

```

00031         if (!out) throw std::runtime_error("Cannot create " + path_);
00032         out << std::fixed << std::setprecision(opt_.decimals);
00033         if (opt_.header) out << "times" << opt_.sep << "prices\n";
00034         for (auto it = begin; it != end; ++it)
00035             out << it->time << opt_.sep << it->price << '\n';
00036     }
00037
00038 protected:
00039     std::string path_;
00040     Options     opt_;
00041 };
00042
00043 } // namespace tp

```

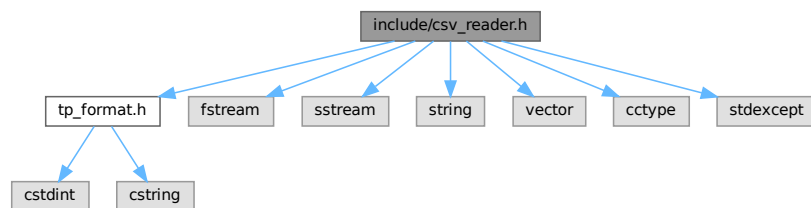
7.9 include/csv_reader.h File Reference

```

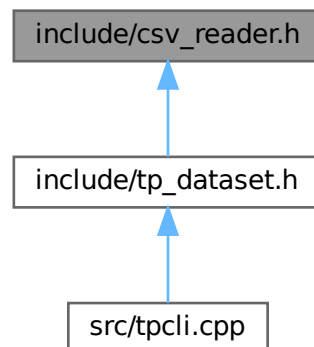
#include "tp_format.h"
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <cctype>
#include <stdexcept>

```

Include dependency graph for csv_reader.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [tp::CsvReader](#)

Namespaces

- namespace [tp](#)

7.10 csv_reader.h

[Go to the documentation of this file.](#)

```

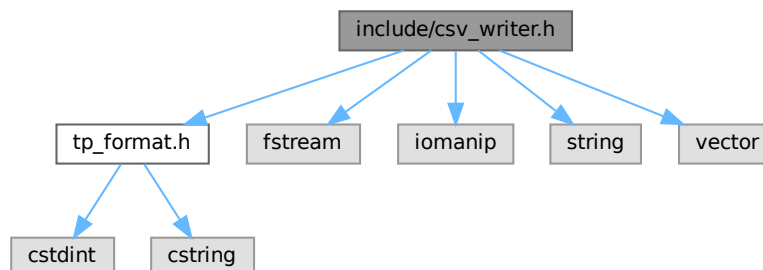
00001 #pragma once
00002 #include "tp_format.h"
00003 #include <fstream>
00004 #include <sstream>
00005 #include <string>
00006 #include <vector>
00007 #include <cctype>
00008 #include <stdexcept>
00009
00010 namespace tp {
00011
00012 class CsvReader {
00013 public:
00014     explicit CsvReader(std::string path) : path_(std::move(path)) {}
00015
00016     std::vector<Tick> load() {
00017         std::ifstream in(path_);
00018         if (!in) throw std::runtime_error("Cannot open CSV: " + path_);
00019
00020         std::vector<Tick> ticks;
00021         std::string line;
00022         bool firstLine = true;
00023         while (std::getline(in, line)) {
00024             if (line.empty()) continue;
00025             if (firstLine) {
00026                 firstLine = false;
00027                 if (!std::isdigit((unsigned char)line[0]) && line[0] != '-' && line[0] != '+')
00028                     continue; // textual header row
00029             }
00030             parseLine(line, ticks);
00031         }
00032         return ticks;
00033     }
00034
00035     int maxDecimals() const { return maxDecimals_; }
00036
00037 private:
00038     void parseLine(const std::string& line, std::vector<Tick>& out) {
00039         auto comma = line.find(',');
00040         if (comma == std::string::npos) return;
00041
00042         Tick t;
00043         t.time = std::stoll(line.substr(0, comma));
00044
00045         std::string pstr = line.substr(comma + 1);
00046         while (!pstr.empty() && std::isspace((unsigned char)pstr.back())) pstr.pop_back();
00047         if (pstr.empty()) return;
00048         t.price = std::stod(pstr);
00049
00050         auto dot = pstr.find('.');
00051         if (dot != std::string::npos) {
00052             int d = 0;
00053             for (size_t i = dot + 1; i < pstr.size() && std::isdigit((unsigned char)pstr[i]); ++i)
00054                 ++d;
00055             if (d > maxDecimals_) maxDecimals_ = d;
00056         }
00057         out.push_back(t);
00058     }
00059
00060     std::string path_;
00061     int maxDecimals_ = 0;
00062 };
00063 } // namespace tp

```

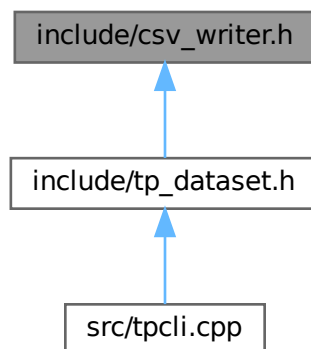
7.11 include/csv_writer.h File Reference

```
#include "tp_format.h"  
#include <fstream>  
#include <iomanip>  
#include <string>  
#include <vector>
```

Include dependency graph for csv_writer.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `tp::CsvWriter`

Namespaces

- namespace `tp`

7.12 csv_writer.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include <fstream>
00004 #include <iomanip>
00005 #include <string>
00006 #include <vector>
00007
00008 namespace tp {
00009
00010 class CsvWriter {
00011 public:
00012     explicit CsvWriter(std::string path, int decimals = 2)
00013         : path_(std::move(path)), decimals_(decimals) {}
00014
00015     void write(const std::vector<Tick>& ticks, bool withHeader = true) const {
00016         std::ofstream out(path_);
00017         if (!out) throw std::runtime_error("Cannot create " + path_);
00018         out << std::fixed << std::setprecision(decimals_);
00019         if (withHeader) out << "times,prices\n";
00020         for (const auto& t : ticks)
00021             out << t.time << ',' << t.price << '\n';
00022     }
00023
00024 private:
00025     std::string path_;
00026     int decimals_;
00027 };
00028
00029 } // namespace tp

```

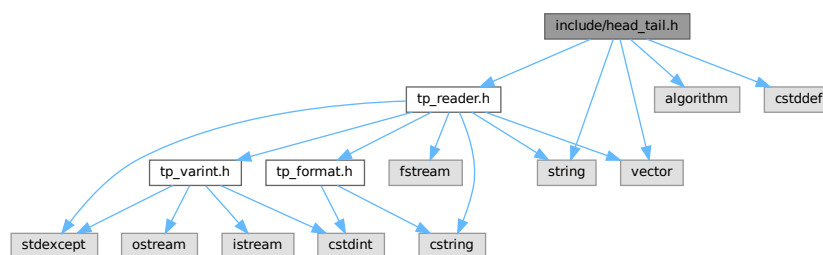
7.13 include/head_tail.h File Reference

```

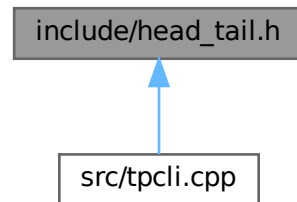
#include "tp_reader.h"
#include <vector>
#include <algorithm>
#include <cstdint>
#include <string>

```

Include dependency graph for head_tail.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `tp::HeadTail`

Namespaces

- namespace `tp`

7.14 head_tail.h

[Go to the documentation of this file.](#)

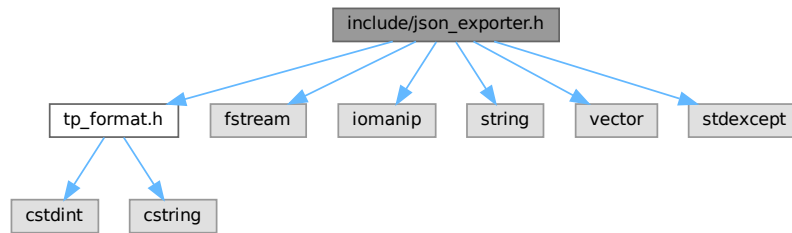
```

00001 #pragma once
00002 #include "tp_reader.h"
00003 #include <vector>
00004 #include <algorithm>
00005 #include <cstdint>
00006 #include <string>
00007
00008 namespace tp {
00009
00010 class HeadTail {
00011 public:
00012     // First N ticks -- O(N), tiny RAM, does NOT load the file.
00013     static std::vector<Tick> head(const std::string& path, size_t n) {
00014         TpReader r(path);
00015         r.open();
00016         std::vector<Tick> out;
00017         out.reserve(n);
00018         Tick t;
00019         while (out.size() < n && r.next(t)) out.push_back(t);
00020         return out;
00021     }
00022
00023     // Last N ticks -- O(total) time, O(N) memory using a ring buffer.
00024     static std::vector<Tick> tail(const std::string& path, size_t n) {
00025         TpReader r(path);
00026         r.open();
00027         std::vector<Tick> ring;
00028         if (n) ring.reserve(n);
00029         size_t idx = 0;
00030         Tick t;
00031         while (r.next(t)) {
00032             if (ring.size() < n) ring.push_back(t);
00033             else { ring[idx] = t; idx = (idx + 1) % n; }
00034         }
00035         if (ring.size() == n && idx != 0)
00036             std::rotate(ring.begin(), ring.begin() + idx, ring.end());
00037         return ring;
00038     }
00039 };
00040
00041 } // namespace tp
  
```

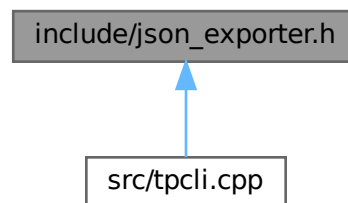
7.15 include/json_exporter.h File Reference

```
#include "tp_format.h"
#include <fstream>
#include <iomanip>
#include <string>
#include <vector>
#include <stdexcept>
```

Include dependency graph for json_exporter.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [tp::JsonExporter](#)
- struct [tp::JsonExporter::Options](#)

Namespaces

- namespace [tp](#)

7.16 json_exporter.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include <fstream>
00004 #include <iomanip>
00005 #include <string>
00006 #include <vector>
00007 #include <stdexcept>
00008
00009 namespace tp {
00010
00011 class JsonExporter {
00012 public:
00013     struct Options {
00014         int decimals = 6;
00015         bool pretty = false;
00016         bool asArrayOfPairs = false;
00017         bool wrapObject = false;
00018     };
00019
00020     explicit JsonExporter(std::string path)
00021         : path_(std::move(path)), opt_{} {}
00022
00023     explicit JsonExporter(std::string path, Options opt)
00024         : path_(std::move(path)), opt_(opt) {}
00025
00026     void write(const std::vector<Tick>& v) const {
00027         std::ofstream out(path_);
00028         if (!out) throw std::runtime_error("Cannot create " + path_);
00029         out << std::fixed << std::setprecision(opt_.decimals);
00030
00031         if (opt_.wrapObject) {
00032             out << "{";
00033             if (opt_.pretty) out << "\n ";
00034             out << "\"count\":" << v.size();
00035             if (!v.empty()) {
00036                 if (opt_.pretty) out << ",\n ";
00037                 out << "\"start_time\":" << v.front().time
00038                     << ", \"end_time\":" << v.back().time;
00039             }
00040             if (opt_.pretty) out << ",\n ";
00041             out << "\"ticks\":";
00042         }
00043         out << "[";
00044         for (size_t i = 0; i < v.size(); ++i) {
00045             if (i) out << ',';
00046             if (opt_.pretty) out << (opt_.asArrayOfPairs ? "\n " : "\n ");
00047             if (opt_.asArrayOfPairs)
00048                 out << "[" << v[i].time << ", " << v[i].price << "]";
00049             else
00050                 out << "{\"time\":" << v[i].time
00051                     << ", \"price\":" << v[i].price << "}";
00052         }
00053         if (opt_.pretty && !v.empty()) out << "\n";
00054         out << "]";
00055         if (opt_.wrapObject) {
00056             if (opt_.pretty) out << "\n";
00057             out << "}";
00058         }
00059         out << "\n";
00060     }
00061
00062 private:
00063     std::string path_;
00064     Options opt_;
00065 };
00066
00067 } // namespace tp

```

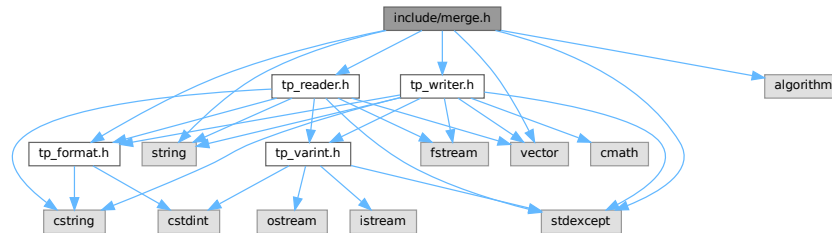
7.17 include/merge.h File Reference

```

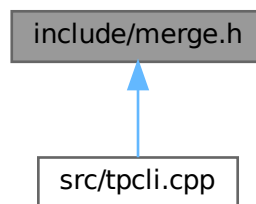
#include "tp_reader.h"
#include "tp_writer.h"
#include "tp_format.h"
#include <string>

```

```
#include <vector>
#include <algorithm>
#include <stdexcept>
Include dependency graph for merge.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [tp::TpMerger](#)
- struct [tp::TpMerger::Options](#)

Namespaces

- namespace [tp](#)

7.18 merge.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include "tp_reader.h"
00003 #include "tp_writer.h"
00004 #include "tp_format.h"
00005 #include <string>
00006 #include <vector>
00007 #include <algorithm>
00008 #include <stdexcept>
00009
```

```

00010 namespace tp {
00011
00012 class TpMerger {
00013 public:
00014     struct Options {
00015         bool dedupe           = true;    // same timestamp -> keep last
00016         bool alreadySorted    = false;   // skip sort if inputs already ordered
00017         int  decimals         = 6;
00018     };
00019
00020     static void mergeToFile(const std::vector<std::string>& inputs,
00021                             const std::string& output)
00022     {
00023         mergeToFile(inputs, output, Options{});
00024     }
00025
00026     static void mergeToFile(const std::vector<std::string>& inputs,
00027                             const std::string& output,
00028                             Options opt)
00029     {
00030         if (inputs.empty()) throw std::runtime_error("no input files");
00031
00032         // Streaming k-way merge by tick count (memory O(total) but simple)
00033         std::vector<Tick> all;
00034         size_t total = 0;
00035         for (auto& p : inputs) {
00036             TpReader r(p); r.open();
00037             total += (size_t)r.header().count;
00038         }
00039         all.reserve(total);
00040
00041         for (auto& p : inputs) {
00042             TpReader r(p); r.open();
00043             Tick t;
00044             while (r.next(t)) all.push_back(t);
00045         }
00046
00047         if (!opt.alreadySorted) {
00048             std::stable_sort(all.begin(), all.end(),
00049                             [](const Tick& a, const Tick& b){ return a.time < b.time; });
00050         }
00051
00052         if (opt.dedupe && all.size() > 1) {
00053             std::vector<Tick> ded;
00054             ded.reserve(all.size());
00055             for (auto& t : all) {
00056                 if (!ded.empty() && ded.back().time == t.time)
00057                     ded.back() = t;    // keep last occurrence
00058                 else
00059                     ded.push_back(t);
00060             }
00061             all.swap(ded);
00062         }
00063
00064         TpWriter(output).write(all, opt.decimals);
00065     }
00066 };
00067
00068 } // namespace tp

```

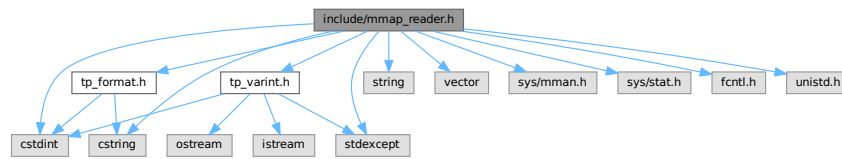
7.19 include/mmap_reader.h File Reference

```

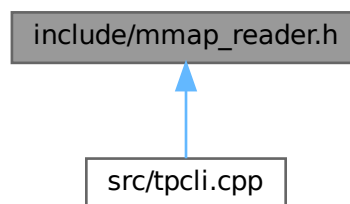
#include "tp_format.h"
#include "tp_varint.h"
#include <string>
#include <vector>
#include <stdexcept>
#include <cstring>
#include <cstdint>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

```

Include dependency graph for `mmap_reader.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class `tp::MmapTpReader`

Namespaces

- namespace `tp`

7.20 mmap_reader.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include "tp_varint.h"
00004 #include <string>
00005 #include <vector>
00006 #include <stdexcept>
00007 #include <cstring>
00008 #include <cstdint>
00009
00010 #include <sys/mman.h>
00011 #include <sys/stat.h>
00012 #include <fcntl.h>
00013 #include <unistd.h>
00014
00015 namespace tp {
00016
00017 // Zero-copy, read-only mmap reader. Ideal for very large .tp files.
00018 class MmapTpReader {
00019 public:
00020     explicit MmapTpReader(std::string path) : path_(std::move(path)) {}

```

```

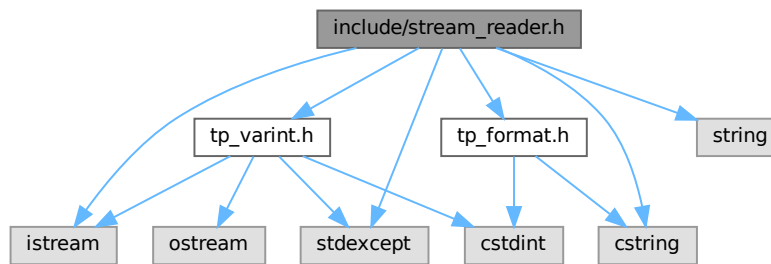
00021 ~MmapTpReader() { close(); }
00022 MmapTpReader(const MmapTpReader&) = delete;
00023 MmapTpReader& operator=(const MmapTpReader&) = delete;
00024
00025 void open() {
00026     fd_ = ::open(path_.c_str(), O_RDONLY);
00027     if (fd_ < 0) throw std::runtime_error("Cannot open " + path_);
00028
00029     struct stat st{};
00030     if (::fstat(fd_, &st) != 0) { ::close(fd_); fd_ = -1; throw std::runtime_error("fstat"); }
00031     size_ = (size_t)st.st_size;
00032     if (size_ < sizeof(TpHeader)) { ::close(fd_); fd_=-1; throw std::runtime_error("Truncated
header"); }
00033
00034     void* p = ::mmap(nullptr, size_, PROT_READ, MAP_PRIVATE, fd_, 0);
00035     if (p == MAP_FAILED) { ::close(fd_); fd_ = -1; throw std::runtime_error("mmap failed"); }
00036     base_ = static_cast<const uint8_t*>(p);
00037
00038     std::memcpy(&header_, base_, sizeof(TpHeader));
00039     if (std::memcmp(header_.magic, TP_MAGIC, 4) != 0)
00040         throw std::runtime_error("Bad magic");
00041     if (header_.version != TP_VERSION)
00042         throw std::runtime_error("Unsupported version");
00043
00044     pos_ = sizeof(TpHeader);
00045     if (header_.count > 0) {
00046         if (pos_ + 16 > size_) throw std::runtime_error("Truncated body");
00047         std::memcpy(&curT_, base_ + pos_, 8); pos_ += 8;
00048         std::memcpy(&curP_, base_ + pos_, 8); pos_ += 8;
00049         idx_ = 1;
00050     }
00051 }
00052
00053 void close() {
00054     if (base_) { ::munmap(const_cast<uint8_t*>(base_), size_); base_ = nullptr; }
00055     if (fd_ >= 0) { ::close(fd_); fd_ = -1; }
00056     size_ = pos_ = 0;
00057 }
00058
00059 const TpHeader& header() const { return header_; }
00060
00061 bool next(Tick& t) {
00062     if (idx_ == 0) {
00063         if (header_.count == 0) return false;
00064         t.time = curT_;
00065         t.price = double(curP_) / double(header_.price_scale);
00066         return true;
00067     }
00068     if (idx_ >= header_.count) return false;
00069     const int64_t dT = zigzagDecode(getVarintMem());
00070     const int64_t dP = zigzagDecode(getVarintMem());
00071     curT_ += dT; curP_ += dP; ++idx_;
00072     t.time = curT_;
00073     t.price = double(curP_) / double(header_.price_scale);
00074     return true;
00075 }
00076
00077 std::vector<Tick> readAll() {
00078     std::vector<Tick> v; v.reserve((size_t)header_.count);
00079     Tick t; while (next(t)) v.push_back(t);
00080     return v;
00081 }
00082
00083 private:
00084     uint64_t getVarintMem() {
00085         uint64_t v = 0; int shift = 0;
00086         while (pos_ < size_) {
00087             uint8_t c = base_[pos_++];
00088             v |= uint64_t(c & 0x7F) << shift;
00089             if (!(c & 0x80)) return v;
00090             shift += 7;
00091             if (shift > 63) throw std::runtime_error("varint overflow");
00092         }
00093         throw std::runtime_error("EOF reading varint");
00094     }
00095
00096     std::string path_;
00097     int fd_ = -1;
00098     const uint8_t* base_ = nullptr;
00099     size_t size_ = 0;
00100     size_t pos_ = 0;
00101     TpHeader header_{};
00102     int64_t curT_ = 0, curP_ = 0;
00103     uint64_t idx_ = 0;
00104 };
00105
00106 } // namespace tp

```

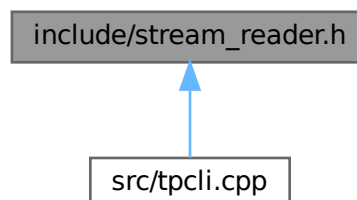
7.21 include/stream_reader.h File Reference

```
#include "tp_format.h"
#include "tp_varint.h"
#include <istream>
#include <string>
#include <stdexcept>
#include <cstring>
```

Include dependency graph for stream_reader.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [tp::StreamTpReader](#)

Namespaces

- namespace [tp](#)

7.22 stream_reader.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include "tp_varint.h"
00004 #include <istream>
00005 #include <string>
00006 #include <stdexcept>
00007 #include <cstring>
00008
00009 namespace tp {
00010
00011 // Pipe-friendly reader: works with std::cin, network stream, etc.
00012 class StreamTpReader {
00013 public:
00014     explicit StreamTpReader(std::istream& in) : in_(in) {}
00015
00016     void start() {
00017         in_.read(reinterpret_cast<char*>(&header_), sizeof(header_));
00018         if (in_.gcount() != (std::streamsize)sizeof(header_))
00019             throw std::runtime_error("Truncated header");
00020         if (std::memcmp(header_.magic, TP_MAGIC, 4) != 0)
00021             throw std::runtime_error("Bad magic");
00022         if (header_.version != TP_VERSION)
00023             throw std::runtime_error("Unsupported version");
00024         if (header_.count > 0) {
00025             in_.read(reinterpret_cast<char*>(&curT_), 8);
00026             in_.read(reinterpret_cast<char*>(&curP_), 8);
00027             if (in_.gcount() != 16) throw std::runtime_error("Truncated body");
00028             idx_ = 1;
00029         }
00030     }
00031
00032     const TpHeader& header() const { return header_; }
00033
00034     bool next(Tick& t) {
00035         if (idx_ == 0) {
00036             if (header_.count == 0) return false;
00037             t.time = curT_; t.price = double(curP_) / double(header_.price_scale);
00038             return true;
00039         }
00040         if (idx_ >= header_.count) return false;
00041         const int64_t dT = zigzagDecode(getVarint(in_));
00042         const int64_t dP = zigzagDecode(getVarint(in_));
00043         curT_ += dT; curP_ += dP; ++idx_;
00044         t.time = curT_; t.price = double(curP_) / double(header_.price_scale);
00045         return true;
00046     }
00047
00048 private:
00049     std::istream& in_;
00050     TpHeader header_;
00051     int64_t curT_ = 0, curP_ = 0;
00052     uint64_t idx_ = 0;
00053 };
00054
00055 } // namespace tp

```

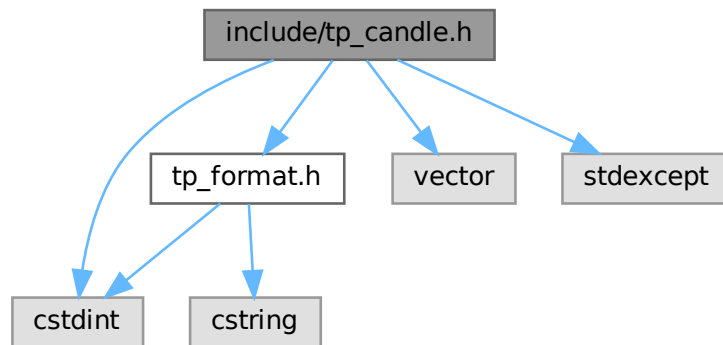
7.23 include/tp_candle.h File Reference

```

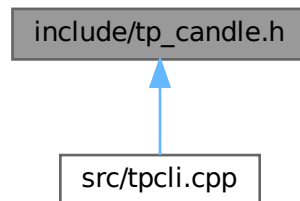
#include "tp_format.h"
#include <vector>
#include <cstdint>
#include <stdexcept>

```

Include dependency graph for `tp_candle.h`:



This graph shows which files directly or indirectly include this file:



Classes

- struct [tp::Candle](#)
- class [tp::CandleBuilder](#)

Namespaces

- namespace [tp](#)

7.24 tp_candle.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include "tp_format.h"
00003 #include <vector>
00004 #include <cstdint>
```

```

00005 #include <stdexcept>
00006
00007 namespace tp {
00008
00009 struct Candle {
00010     int64_t start_time; // bucket start (aligned)
00011     int64_t end_time;   // bucket start + interval (exclusive)
00012     double  open, high, low, close;
00013     uint64_t volume;    // tick count in this candle
00014 };
00015
00016 class CandleBuilder {
00017 public:
00018     // intervalSeconds > 0. Buckets are aligned to EPOCH: t/interval*interval
00019     static std::vector<Candle> build(const std::vector<Tick>& ticks, int64_t intervalSeconds) {
00020         if (intervalSeconds <= 0) throw std::runtime_error("interval must be > 0");
00021         std::vector<Candle> out;
00022         if (ticks.empty()) return out;
00023
00024         int64_t bucket = alignDown(ticks.front().time, intervalSeconds);
00025         Candle c{};
00026         c.start_time = bucket;
00027         c.end_time   = bucket + intervalSeconds;
00028         c.open = c.high = c.low = c.close = ticks.front().price;
00029         c.volume = 0;
00030
00031         for (const auto& t : ticks) {
00032             int64_t b = alignDown(t.time, intervalSeconds);
00033             if (b != bucket) {
00034                 out.push_back(c);
00035                 bucket = b;
00036                 c = Candle{};
00037                 c.start_time = bucket;
00038                 c.end_time   = bucket + intervalSeconds;
00039                 c.open = c.high = c.low = c.close = t.price;
00040                 c.volume = 0;
00041             }
00042             if (t.price > c.high) c.high = t.price;
00043             if (t.price < c.low)  c.low  = t.price;
00044             c.close = t.price;
00045             ++c.volume;
00046         }
00047         out.push_back(c);
00048         return out;
00049     }
00050
00051 private:
00052     static int64_t alignDown(int64_t t, int64_t step) {
00053         int64_t r = t % step;
00054         if (r < 0) r += step;
00055         return t - r;
00056     }
00057 };
00058
00059 } // namespace tp

```

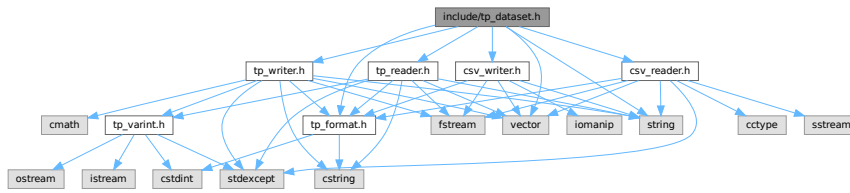
7.25 include/tp_dataset.h File Reference

```

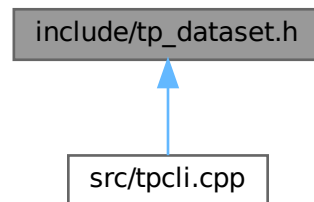
#include "tp_format.h"
#include "csv_reader.h"
#include "csv_writer.h"
#include "tp_reader.h"
#include "tp_writer.h"
#include <string>
#include <vector>

```

Include dependency graph for `tp_dataset.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class [tp::TpDataset](#)

Namespaces

- namespace [tp](#)

7.26 tp_dataset.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include "csv_reader.h"
00004 #include "csv_writer.h"
00005 #include "tp_reader.h"
00006 #include "tp_writer.h"
00007 #include <string>
00008 #include <vector>
00009
00010 namespace tp {
00011
00012 // Façade: loads either CSV or .tp into RAM, lets you save to either form.
00013 class TpDataset {
00014 public:
00015     static TpDataset fromCsv(const std::string& path) {
00016         CsvReader r(path);
00017         TpDataset d;
00018         d.ticks_ = r.load();
00019         d.decimals_ = r.maxDecimals();
00020     }
00021 };
  
```

```

00020         d.rebuildHeader();
00021         return d;
00022     }
00023
00024     static TpDataset fromTp(const std::string& path) {
00025         TpReader r(path);
00026         r.open();
00027         TpDataset d;
00028         d.header_ = r.header();
00029         d.decimals_ = decimalsFromScale(d.header_.price_scale);
00030         d.ticks_ = r.readAll();
00031         return d;
00032     }
00033
00034     void toCsv(const std::string& path) const {
00035         CsvWriter(path, decimals_).write(ticks_);
00036     }
00037     void toTp(const std::string& path) const {
00038         TpWriter(path).write(ticks_, decimals_);
00039     }
00040
00041     const TpHeader& header() const { return header_; }
00042     const std::vector<Tick>& ticks() const { return ticks_; }
00043     size_t size() const { return ticks_.size(); }
00044     bool empty() const { return ticks_.empty(); }
00045     int decimals() const { return decimals_; }
00046
00047 private:
00048     static int decimalsFromScale(uint32_t s) {
00049         int d = 0; while (s > 1 && s % 10 == 0) { s /= 10; ++d; }
00050         return d;
00051     }
00052
00053     void rebuildHeader() {
00054         header_ = TpHeader{};
00055         std::memcpy(header_.magic, TP_MAGIC, 4);
00056         header_.version = TP_VERSION;
00057         header_.flags = TP_FLAG_COMPRESSED;
00058         header_.count = ticks_.size();
00059         header_.price_scale = (uint32_t)std::pow(10u, decimals_);
00060         if (!ticks_.empty()) {
00061             header_.start_time = ticks_.front().time;
00062             header_.end_time = ticks_.back().time;
00063             double mn = ticks_.front().price, mx = mn;
00064             for (const auto& t : ticks_) { mn = std::min(mn, t.price); mx = std::max(mx, t.price); }
00065             header_.min_price = mn;
00066             header_.max_price = mx;
00067         }
00068     }
00069
00070     TpHeader header_;
00071     std::vector<Tick> ticks_;
00072     int decimals_ = 2;
00073 };
00074
00075 } // namespace tp

```

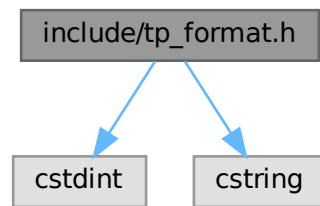
7.27 include/tp_format.h File Reference

```

#include <cstdint>
#include <cstring>

```

Include dependency graph for `tp_format.h`:



This graph shows which files directly or indirectly include this file:



Classes

- struct `tp::TpHeader`
- struct `tp::Tick`

Namespaces

- namespace `tp`

Variables

- constexpr char `tp::TP_MAGIC` [4] = {'T','P','0','1'}
- constexpr uint32_t `tp::TP_VERSION` = 1
- constexpr uint32_t `tp::TP_FLAG_COMPRESSED` = 1u << 0

7.28 tp_format.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include <cstdint>
00003 #include <cstring>
00004
00005 namespace tp {
00006
00007     constexpr char    TP_MAGIC[4]      = {'T','P','0','1'};
00008     constexpr uint32_t TP_VERSION      = 1;
00009     constexpr uint32_t TP_FLAG_COMPRESSED = 1u << 0;
00010
00011     #pragma pack(push,1)
00012     struct TpHeader {

```

```

00013     char    magic[4];        // "TP01"
00014     uint32_t version;        // 1
00015     uint32_t flags;          // bit0 = compressed
00016     int64_t  start_time;     // unix seconds of first tick
00017     int64_t  end_time;       // unix seconds of last tick
00018     double   min_price;
00019     double   max_price;
00020     uint64_t count;          // number of ticks
00021     uint32_t price_scale;    // 10^decimals
00022     uint32_t reserved;      // must be 0
00023 };
00024 #pragma pack(pop)
00025
00026 static_assert(sizeof(TpHeader) == 60, "TpHeader must be 60 bytes");
00027
00028 struct Tick {
00029     int64_t time;            // unix seconds
00030     double price;
00031 };
00032
00033 } // namespace tp

```

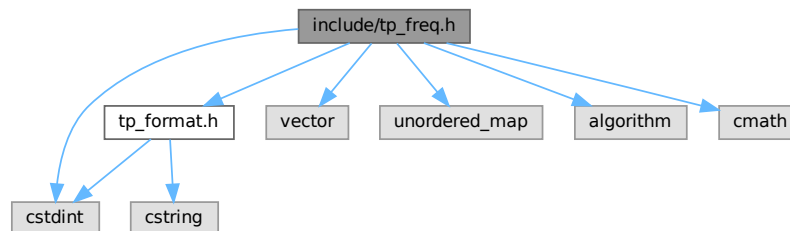
7.29 include/tp_freq.h File Reference

```

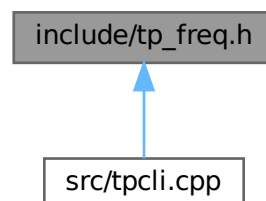
#include "tp_format.h"
#include <vector>
#include <unordered_map>
#include <algorithm>
#include <cmath>
#include <cstdint>

```

Include dependency graph for tp_freq.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [tp::FreqEntry](#)
- class [tp::FrequencyAnalyzer](#)

Namespaces

- namespace [tp](#)

7.30 tp_freq.h

[Go to the documentation of this file.](#)

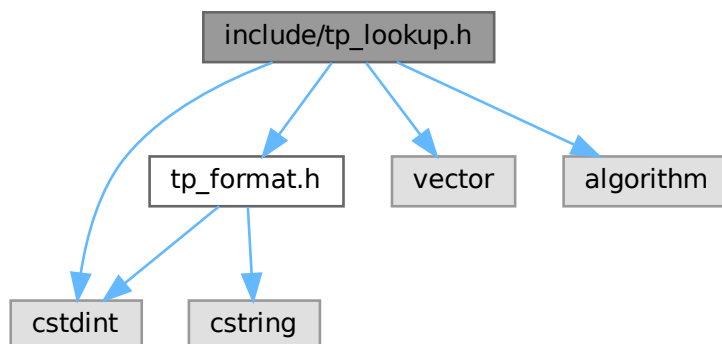
```
00001 #pragma once
00002 #include "tp_format.h"
00003 #include <vector>
00004 #include <unordered_map>
00005 #include <algorithm>
00006 #include <cmath>
00007 #include <cstdint>
00008
00009 namespace tp {
00010
00011 struct FreqEntry {
00012     double price;
00013     uint64_t count;
00014 };
00015
00016 class FrequencyAnalyzer {
00017 public:
00018     // Group by scaled integer (avoids floating point key issues)
00019     static std::vector<FreqEntry> compute(const std::vector<Tick>& ticks, uint32_t scale) {
00020         std::unordered_map<int64_t, uint64_t> hist;
00021         hist.reserve(ticks.size());
00022         for (const auto& t : ticks) {
00023             int64_t k = (int64_t)std::llround(t.price * (double)scale);
00024             ++hist[k];
00025         }
00026         std::vector<FreqEntry> out;
00027         out.reserve(hist.size());
00028         for (auto& kv : hist)
00029             out.push_back({ double(kv.first) / double(scale), kv.second });
00030         return out;
00031     }
00032
00033     // sort by count descending, then price ascending
00034     static std::vector<FreqEntry> sortByCountDesc(std::vector<FreqEntry> v) {
00035         std::sort(v.begin(), v.end(), [](const FreqEntry& a, const FreqEntry& b){
00036             if (a.count != b.count) return a.count > b.count;
00037             return a.price < b.price;
00038         });
00039         return v;
00040     }
00041
00042     // sort by price ascending
00043     static std::vector<FreqEntry> sortByPrice(std::vector<FreqEntry> v) {
00044         std::sort(v.begin(), v.end(), [](const FreqEntry& a, const FreqEntry& b){
00045             return a.price < b.price;
00046         });
00047         return v;
00048     }
00049 };
00050
00051 } // namespace tp
```

7.31 include/tp_lookup.h File Reference

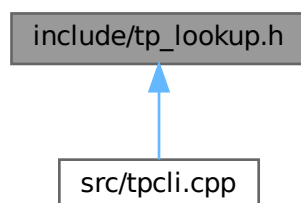
```
#include "tp_format.h"
#include <vector>
#include <algorithm>
```

```
#include <cstdint>
```

Include dependency graph for tp_lookup.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `tp::PriceLookup`

Namespaces

- namespace `tp`

7.32 tp_lookup.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include "tp_format.h"
00003 #include <vector>
00004 #include <algorithm>
```

```

00005 #include <cstdint>
00006
00007 namespace tp {
00008
00009 class PriceLookup {
00010 public:
00011     // exact match on time; nullptr if not present
00012     static const Tick* exact(const std::vector<Tick>& v, int64_t time) {
00013         auto it = std::lower_bound(v.begin(), v.end(), time,
00014             [](const Tick& t, int64_t s){ return t.time < s; });
00015         if (it != v.end() && it->time == time) return &*it;
00016         return nullptr;
00017     }
00018
00019     // nearest tick (by |time diff|)
00020     static const Tick* nearest(const std::vector<Tick>& v, int64_t time) {
00021         if (v.empty()) return nullptr;
00022         auto it = std::lower_bound(v.begin(), v.end(), time,
00023             [](const Tick& t, int64_t s){ return t.time < s; });
00024         if (it == v.begin()) return &v.front();
00025         if (it == v.end()) return &v.back();
00026         const Tick* a = &*(it - 1);
00027         const Tick* b = &*it;
00028         return (time - a->time) <= (b->time - time) ? a : b;
00029     }
00030
00031     // last tick with time <= given (as-of)
00032     static const Tick* atOrBefore(const std::vector<Tick>& v, int64_t time) {
00033         if (v.empty()) return nullptr;
00034         auto it = std::upper_bound(v.begin(), v.end(), time,
00035             [](int64_t s, const Tick& t){ return s < t.time; });
00036         if (it == v.begin()) return nullptr;
00037         return &*(it - 1);
00038     }
00039
00040     // [t1, t2] inclusive
00041     static std::vector<Tick> range(const std::vector<Tick>& v, int64_t t1, int64_t t2) {
00042         std::vector<Tick> out;
00043         auto lo = std::lower_bound(v.begin(), v.end(), t1,
00044             [](const Tick& t, int64_t s){ return t.time < s; });
00045         for (auto it = lo; it != v.end() && it->time <= t2; ++it) out.push_back(*it);
00046         return out;
00047     }
00048 };
00049
00050 } // namespace tp

```

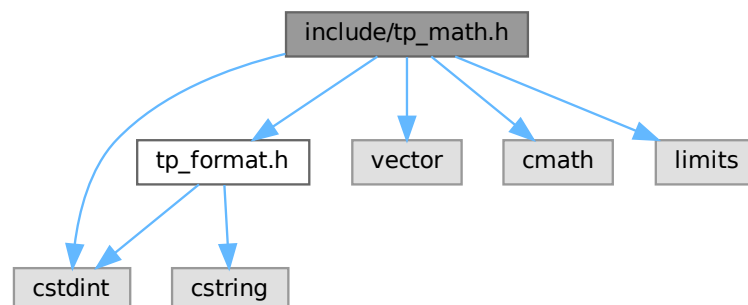
7.33 include/tp_math.h File Reference

```

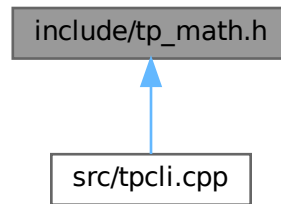
#include "tp_format.h"
#include <vector>
#include <cmath>
#include <limits>
#include <cstdint>

```

Include dependency graph for tp_math.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct `tp::MathResult`
- class `tp::MathStats`

Namespaces

- namespace `tp`

7.34 tp_math.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include <vector>
00004 #include <cmath>
00005 #include <limits>
00006 #include <cstdint>
00007
00008 namespace tp {
00009
00010 struct MathResult {
00011     uint64_t count = 0;
00012     int64_t min_time = 0;
00013     int64_t max_time = 0;
00014     double min_price = 0;
00015     double max_price = 0;
00016     double sum = 0;
00017     double average = 0;
00018     long double product = 1.0L; // can overflow for large N
00019     bool productOverflow = false;
00020     double geometricMean = 0; // exp(mean(log(p)))
00021     double sumTimesCount = 0; // sum * count (cheap helper)
00022 };
00023
00024 class MathStats {
00025 public:
00026     static MathResult compute(const std::vector<Tick>& ticks) {
00027         MathResult r;
00028         if (ticks.empty()) return r;
00029
00030         r.count = ticks.size();
00031         r.min_time = ticks.front().time;
00032         r.max_time = ticks.back().time;
00033         r.min_price = ticks.front().price;
00034         r.max_price = ticks.front().price;
00035
00036         long double logSum = 0.0L;
00037         for (const auto& t : ticks) {

```

```

00038         r.sum += t.price;
00039         if (t.price < r.min_price) r.min_price = t.price;
00040         if (t.price > r.max_price) r.max_price = t.price;
00041
00042         if (!r.productOverflow) {
00043             r.product *= (long double)t.price;
00044             if (!std::isfinite((double)r.product)) r.productOverflow = true;
00045         }
00046         if (t.price > 0) logSum += std::log((long double)t.price);
00047     }
00048     r.average = r.sum / double(r.count);
00049     r.geometricMean = (double)std::exp(logSum / (long double)r.count);
00050     r.sumTimesCount = r.sum * (double)r.count;
00051     return r;
00052 }
00053 };
00054
00055 } // namespace tp

```

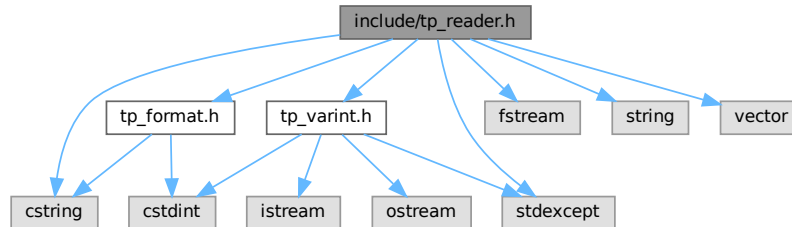
7.35 include/tp_reader.h File Reference

```

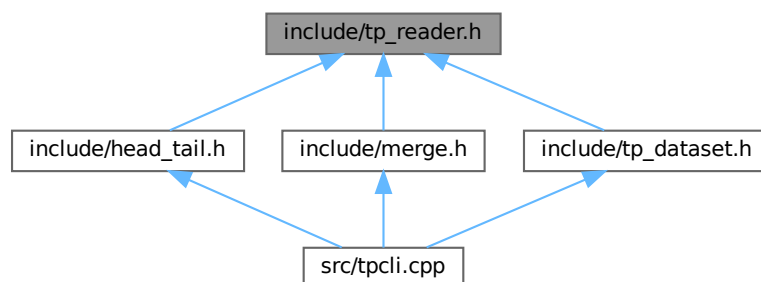
#include "tp_format.h"
#include "tp_varint.h"
#include <fstream>
#include <cstring>
#include <string>
#include <vector>
#include <stdexcept>

```

Include dependency graph for tp_reader.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `tp::TpReader`

Namespaces

- namespace `tp`

7.36 tp_reader.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include "tp_varint.h"
00004 #include <fstream>
00005 #include <cstring>
00006 #include <string>
00007 #include <vector>
00008 #include <stdexcept>
00009
00010 namespace tp {
00011
00012 class TpReader {
00013 public:
00014     explicit TpReader(std::string path) : path_(std::move(path)) {}
00015
00016     void open() {
00017         in_.open(path_, std::ios::binary);
00018         if (!in_) throw std::runtime_error("Cannot open " + path_);
00019         in_.read(reinterpret_cast<char*>(&header_), sizeof(header_));
00020         if (in_.gcount() != (std::streamsize)sizeof(header_))
00021             throw std::runtime_error("Truncated .tp header: " + path_);
00022         if (std::memcmp(header_.magic, TP_MAGIC, 4) != 0)
00023             throw std::runtime_error("Bad magic - not a .tp file");
00024         if (header_.version != TP_VERSION)
00025             throw std::runtime_error("Unsupported .tp version");
00026
00027         if (header_.count > 0) {
00028             in_.read(reinterpret_cast<char*>(&curT_), sizeof(curT_));
00029             in_.read(reinterpret_cast<char*>(&curP_), sizeof(curP_));
00030             if (in_.gcount() != (std::streamsize)sizeof(curP_))
00031                 throw std::runtime_error("Truncated .tp body");
00032             idx_ = 1;
00033         }
00034     }
00035
00036     const TpHeader& header() const { return header_; }
00037
00038     bool next(Tick& outTick) {
00039         if (idx_ == 0) {
00040             if (header_.count == 0) return false;
00041             outTick.time = curT_;
00042             outTick.price = double(curP_) / double(header_.price_scale);
00043             return true;
00044         }
00045         if (idx_ >= header_.count) return false;
00046
00047         const int64_t dT = zigzagDecode(getVarint(in_));
00048         const int64_t dP = zigzagDecode(getVarint(in_));
00049         curT_ += dT; curP_ += dP; ++idx_;
00050         outTick.time = curT_;
00051         outTick.price = double(curP_) / double(header_.price_scale);
00052         return true;
00053     }
00054
00055     std::vector<Tick> readAll() {
00056         std::vector<Tick> v;
00057         v.reserve((size_t)header_.count);
00058         Tick t;
00059         while (next(t)) v.push_back(t);
00060         return v;
00061     }
00062
00063 private:
00064     std::string path_;
00065     std::ifstream in_;

```

```

00066     TpHeader header_{};
00067     int64_t curT_ = 0, curP_ = 0;
00068     uint64_t idx_ = 0;
00069 };
00070
00071 } // namespace tp

```

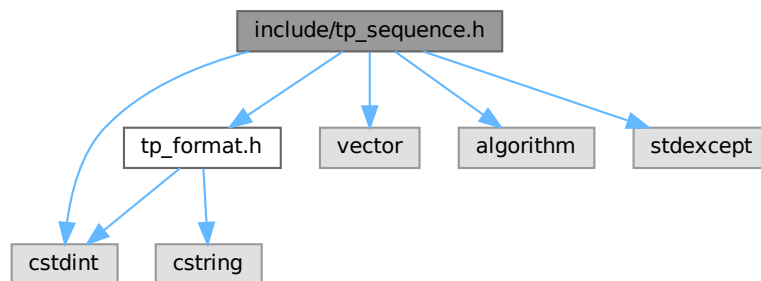
7.37 include/tp_sequence.h File Reference

```

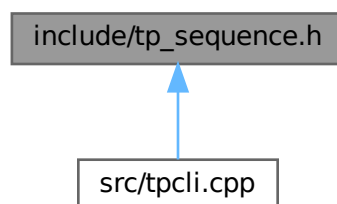
#include "tp_format.h"
#include <vector>
#include <algorithm>
#include <cstdint>
#include <stdexcept>

```

Include dependency graph for tp_sequence.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `tp::Sequencer`

Namespaces

- namespace `tp`

7.38 tp_sequence.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include "tp_format.h"
00003 #include <vector>
00004 #include <algorithm>
00005 #include <cstdint>
00006 #include <stdexcept>
00007
00008 namespace tp {
00009
00010 class Sequencer {
00011 public:
00012     // index based: 0, N, 2N, 3N ...
00013     static std::vector<Tick> everyNthTick(const std::vector<Tick>& v, size_t n) {
00014         if (n == 0) throw std::runtime_error("N must be >= 1");
00015         std::vector<Tick> out;
00016         out.reserve((v.size() + n - 1) / n);
00017         for (size_t i = 0; i < v.size(); i += n) out.push_back(v[i]);
00018         return out;
00019     }
00020
00021     // time based: at t0, t0+N, t0+2N, ... pick tick nearest to each sample time
00022     static std::vector<Tick> everyNSeconds(const std::vector<Tick>& v, int64_t n) {
00023         if (n <= 0) throw std::runtime_error("N seconds must be >= 1");
00024         std::vector<Tick> out;
00025         if (v.empty()) return out;
00026
00027         int64_t t = v.front().time;
00028         int64_t tEnd = v.back().time;
00029         while (t <= tEnd) {
00030             const Tick* nearest = nullptr;
00031             auto it = std::lower_bound(v.begin(), v.end(), t,
00032                                     [](const Tick& x, int64_t s){ return x.time < s; });
00033             if (it == v.begin()) nearest = &v.front();
00034             else if (it == v.end()) nearest = &v.back();
00035             else {
00036                 const Tick* a = &*(it - 1);
00037                 const Tick* b = &*it;
00038                 nearest = (t - a->time) <= (b->time - t) ? a : b;
00039             }
00040             out.push_back(*nearest);
00041             t += n;
00042         }
00043         return out;
00044     }
00045 };
00046
00047 } // namespace tp

```

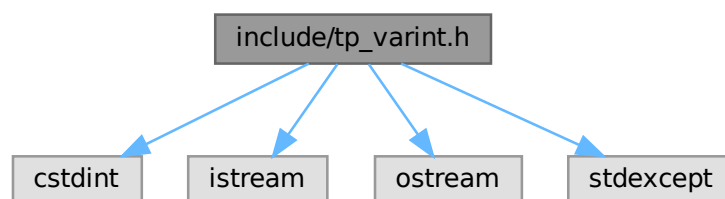
7.39 include/tp_varint.h File Reference

```

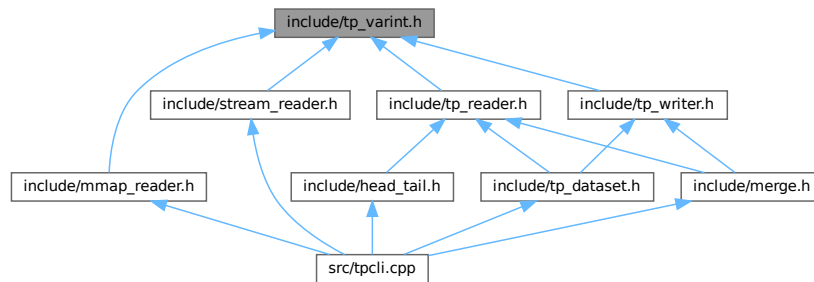
#include <cstdint>
#include <istream>
#include <ostream>
#include <stdexcept>

```

Include dependency graph for tp_varint.h:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace `tp`

Functions

- void `tp::putVarint` (std::ostream &os, uint64_t v)
- uint64_t `tp::getVarint` (std::istream &is)
- uint64_t `tp::zigzagEncode` (int64_t v)
- int64_t `tp::zigzagDecode` (uint64_t v)

7.40 tp_varint.h

[Go to the documentation of this file.](#)

```

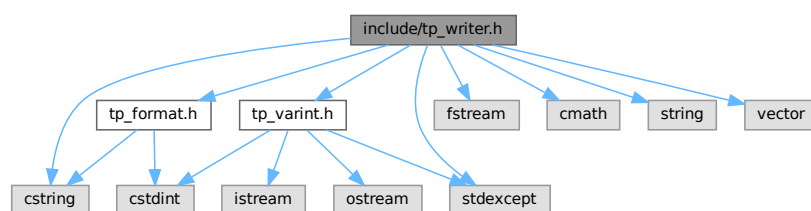
00001 #pragma once
00002 #include <stdint>
00003 #include <istream>
00004 #include <ostream>
00005 #include <stdexcept>
00006
00007 namespace tp {
00008
00009 inline void putVarint(std::ostream& os, uint64_t v) {
00010     while (v >= 0x80) { os.put(char((v & 0x7F) | 0x80)); v >>= 7; }
00011     os.put(char(v));
00012 }
00013 inline uint64_t getVarint(std::istream& is) {
00014     uint64_t v = 0; int shift = 0;
00015     for (;;) {
00016         int c = is.get();
00017         if (c < 0) throw std::runtime_error("Unexpected EOF while reading varint");
00018         v |= uint64_t(c & 0x7F) << shift;
00019         if (!(c & 0x80)) break;
00020         shift += 7;
00021         if (shift > 63) throw std::runtime_error("varint overflow");
00022     }
00023     return v;
00024 }
00025 inline uint64_t zigzagEncode(int64_t v) { return (uint64_t(v) << 1) ^ uint64_t(v >> 63); }
00026 inline int64_t zigzagDecode(uint64_t v) { return int64_t(v >> 1) ^ -int64_t(v & 1); }
00027
00028 } // namespace tp

```

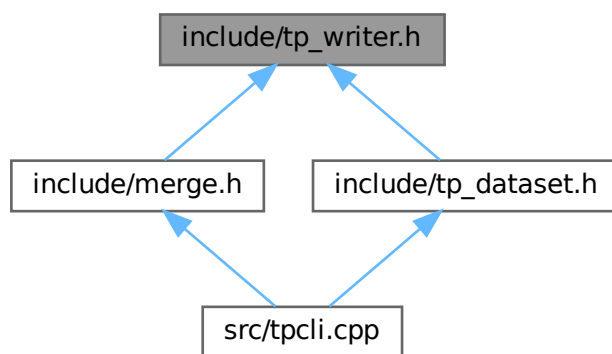
7.41 include/tp_writer.h File Reference

```
#include "tp_format.h"
#include "tp_varint.h"
#include <fstream>
#include <cmath>
#include <cstring>
#include <string>
#include <vector>
#include <stdexcept>
```

Include dependency graph for tp_writer.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [tp::TpWriter](#)

Namespaces

- namespace [tp](#)

7.42 tp_writer.h

[Go to the documentation of this file.](#)

```

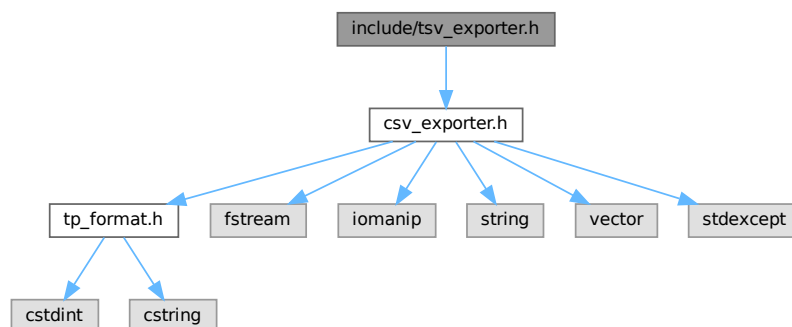
00001 #pragma once
00002 #include "tp_format.h"
00003 #include "tp_varint.h"
00004 #include <fstream>
00005 #include <cmath>
00006 #include <cstring>
00007 #include <string>
00008 #include <vector>
00009 #include <stdexcept>
00010
00011 namespace tp {
00012
00013 class TpWriter {
00014 public:
00015     explicit TpWriter(std::string path) : path_(std::move(path)) {}
00016
00017     void write(const std::vector<Tick>& ticks, int decimals = 2) const {
00018         if (ticks.empty()) throw std::runtime_error("No ticks to write");
00019         if (decimals < 0) decimals = 0;
00020         if (decimals > 9) decimals = 9;
00021         const uint32_t scale = (uint32_t)std::pow(10u, decimals);
00022
00023         TpHeader h{};
00024         std::memcpy(h.magic, TP_MAGIC, 4);
00025         h.version = TP_VERSION;
00026         h.flags = TP_FLAG_COMPRESSED;
00027         h.start_time = ticks.front().time;
00028         h.end_time = ticks.back().time;
00029         h.count = ticks.size();
00030         h.price_scale = scale;
00031
00032         double mn = ticks.front().price, mx = ticks.front().price;
00033         for (const auto& t : ticks) {
00034             if (t.price < mn) mn = t.price;
00035             if (t.price > mx) mx = t.price;
00036         }
00037         h.min_price = mn;
00038         h.max_price = mx;
00039
00040         std::ofstream out(path_, std::ios::binary);
00041         if (!out) throw std::runtime_error("Cannot create " + path_);
00042
00043         out.write(reinterpret_cast<const char*>(&h), sizeof(h));
00044
00045         int64_t prevT = ticks.front().time;
00046         int64_t prevP = (int64_t)std::llround(ticks.front().price * (double)scale);
00047         out.write(reinterpret_cast<const char*>(&prevT), sizeof(prevT));
00048         out.write(reinterpret_cast<const char*>(&prevP), sizeof(prevP));
00049
00050         for (size_t i = 1; i < ticks.size(); ++i) {
00051             int64_t curT = ticks[i].time;
00052             int64_t curP = (int64_t)std::llround(ticks[i].price * (double)scale);
00053             putVarint(out, zigzagEncode(curT - prevT));
00054             putVarint(out, zigzagEncode(curP - prevP));
00055             prevT = curT;
00056             prevP = curP;
00057         }
00058     }
00059
00060 private:
00061     std::string path_;
00062 };
00063
00064 } // namespace tp

```

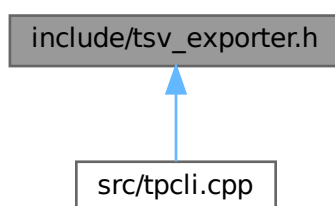
7.43 include/tsv_exporter.h File Reference

```
#include "csv_exporter.h"
```

Include dependency graph for tsv_exporter.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `tp::TsvExporter`

Namespaces

- namespace `tp`

7.44 tsv_exporter.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include "csv_exporter.h"
00003
```

```

00004 namespace tp {
00005
00006 class TsvExporter : public CsvExporter {
00007 public:
00008     explicit TsvExporter(std::string path, Options opt = {})
00009         : CsvExporter(std::move(path), fix(opt)) {}
00010
00011 private:
00012     static Options fix(Options o) { o.sep = '\\t'; return o; }
00013 };
00014
00015 } // namespace tp

```

7.45 src/tpcli.cpp File Reference

```

#include "tp_dataset.h"
#include "tp_math.h"
#include "tp_candle.h"
#include "tp_lookup.h"
#include "tp_freq.h"
#include "tp_sequence.h"
#include "csv_exporter.h"
#include "tsv_exporter.h"
#include "json_exporter.h"
#include "columnar_exporter.h"
#include "arrow_parquet_exporter.h"
#include "head_tail.h"
#include "bucket_stats.h"
#include "merge.h"
#include "mmap_reader.h"
#include "stream_reader.h"
#include <iostream>
#include <iomanip>
#include <string>
#include <cstring>
#include <chrono>

```

Include dependency graph for tpcli.cpp:



Functions

- static void `printHelp` (const char *prog)
- static void `printHeader` (const `TpHeader` &h)
- int `main` (int argc, char **argv)

Variables

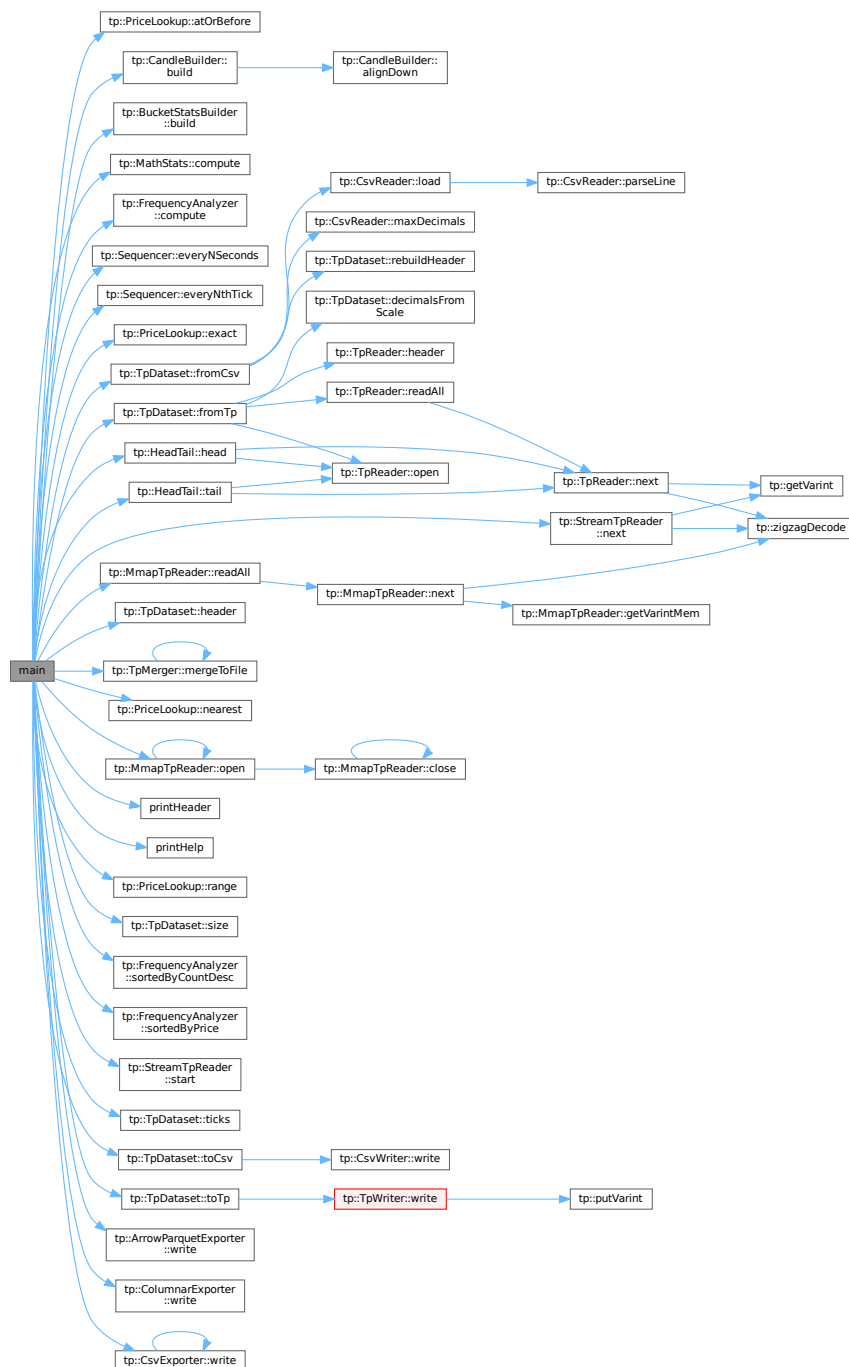
- static const char * `VERSION` = "tpcli 1.0.0 (TP Time-Price codec)"

7.45.1 Function Documentation

7.45.1.1 main()

```
int main (
    int argc,
    char ** argv )
```

Here is the call graph for this function:



7.45.1.2 printHeader()

```
static void printHeader (
    const TpHeader & h ) [static]
```

Here is the caller graph for this function:



7.45.1.3 printHelp()

```
static void printHelp (
    const char * prog ) [static]
```

Here is the caller graph for this function:



7.45.2 Variable Documentation

7.45.2.1 VERSION

```
const char* VERSION = "tpcli 1.0.0 (TP Time-Price codec)" [static]
```